

Associate Architect 양성과정 · Architecture 설계

Day 4 · Architectural Tactics (2) & Design

Testability / Usability Tactics · Tactics and Patterns · Architecture Design (ADD)

- 3.10 Testability · 3.11 Usability Tactics
- 3.12 Tactics and Architectural Patterns
- 4. Architecture Design (ADD, FCAPS 사례연구)

3.10

Testability Tactics

테스트를 통해 소프트웨어의 결함을 쉽게 드러낼 수 있게 만드는 تاک

- Testability 의 정의
- Testability Scenario
- Control and Observe System State
- Limit Complexity

Testability(테스트 용이성)란

■ Testability 의 정의

- 테스트를 통해 소프트웨어가 자신의 결함(fault)을 얼마나 쉽게 드러내도록 만들 수 있는가
- 소프트웨어에 결함이 최소 하나 있다고 가정할 때, 다음 테스트 실행에서 실패할(결함이 드러날) 확률

■ Testability 의 Response Measure 가 다루는 것

- 테스트가 결함을 찾아내는 데 얼마나 효과적인가
- 원하는 수준의 coverage 까지 테스트하는 데 시간이 얼마나 걸리는가

핵심 테스트하기 쉬운 시스템 = 결함이 있으면 테스트에서 잘 드러나는 시스템

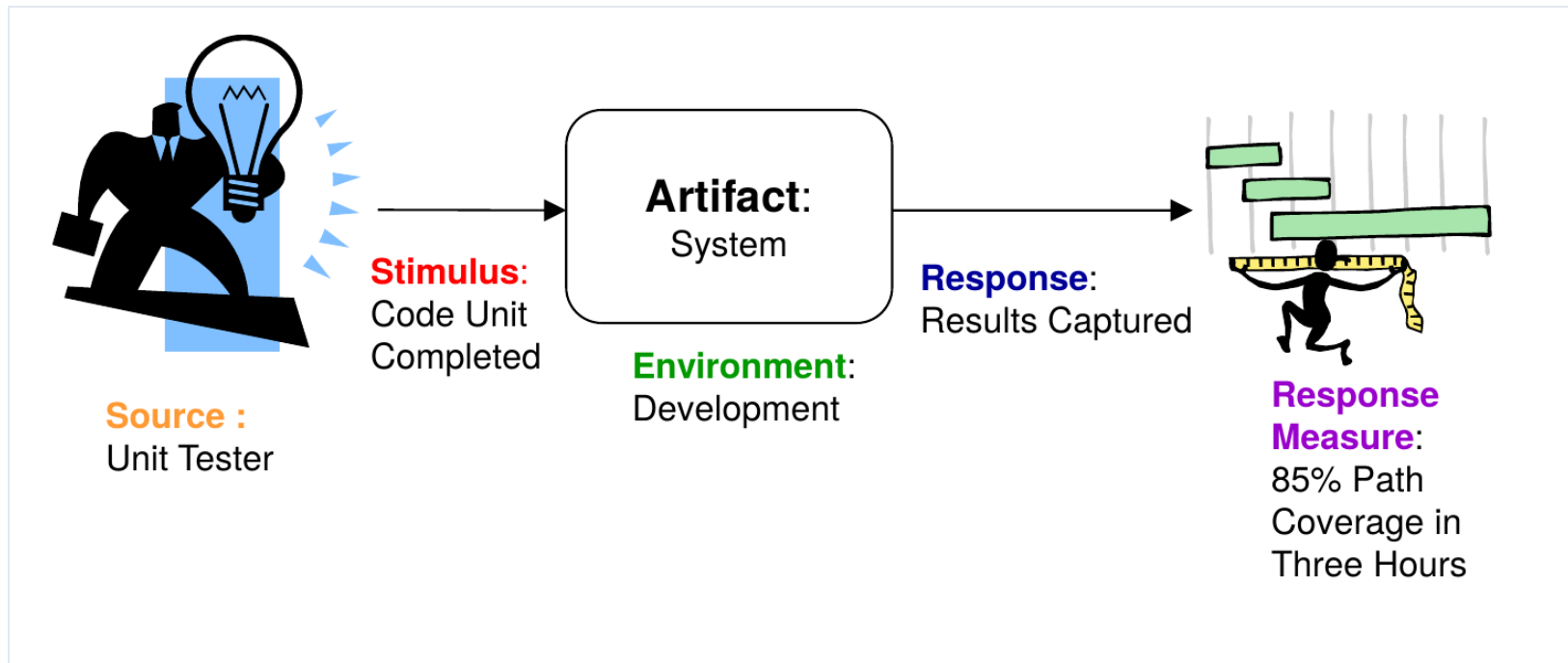
General Testability Scenario

부분	가능한 값
Source	사람(unit tester, integration tester, system tester, acceptance tester, 최종 사용자) / 자동화된 tester
Stimulus	다음 시점에 테스트 집합이 실행된다: class·layer·service 같은 코딩 증분 완료 / 서브시스템 통합 완료 / 전체 시스템 구현 완료 / 고객에게 시스템 인도
Artifact	테스트 대상이 되는 시스템의 부분
Environment	설계 시점, 개발 시점, 컴파일 시점, 통합 시점, 배포 시점, 실행 시점
Response	test suite 를 실행하고 결과를 수집 / 결함을 일으킨 활동을 수집 / 시스템 상태를 제어하고 관찰
Response Measure	결함(또는 결함 부류)을 찾는 노력 / 주어진 비율의 state space coverage 를 달성하는 노력 / 다음 테스트에서 결함이 드러날 확률 / 테스트 수행 시간 / 결함 감지 노력, 테스트의 최장 의존 체인 길이, 테스트 환경 준비 시간 / 위험 노출 감소량 (size(loss) × prob(loss))

Concrete Testability Scenario 예

■ 시나리오

- "unit tester 가 개발 중 코드 단위를 완성하고 테스트 시퀀스를 수행한다. 결과가 수집되며, 3시간의 테스트로 85% path coverage 를 얻는다"



Source: Unit Tester / Stimulus: 코드 단위 완성 / Artifact: System / Environment: 개발 시점 / Response: 결과 수집 / Response Measure: 3시간에 85% path coverage

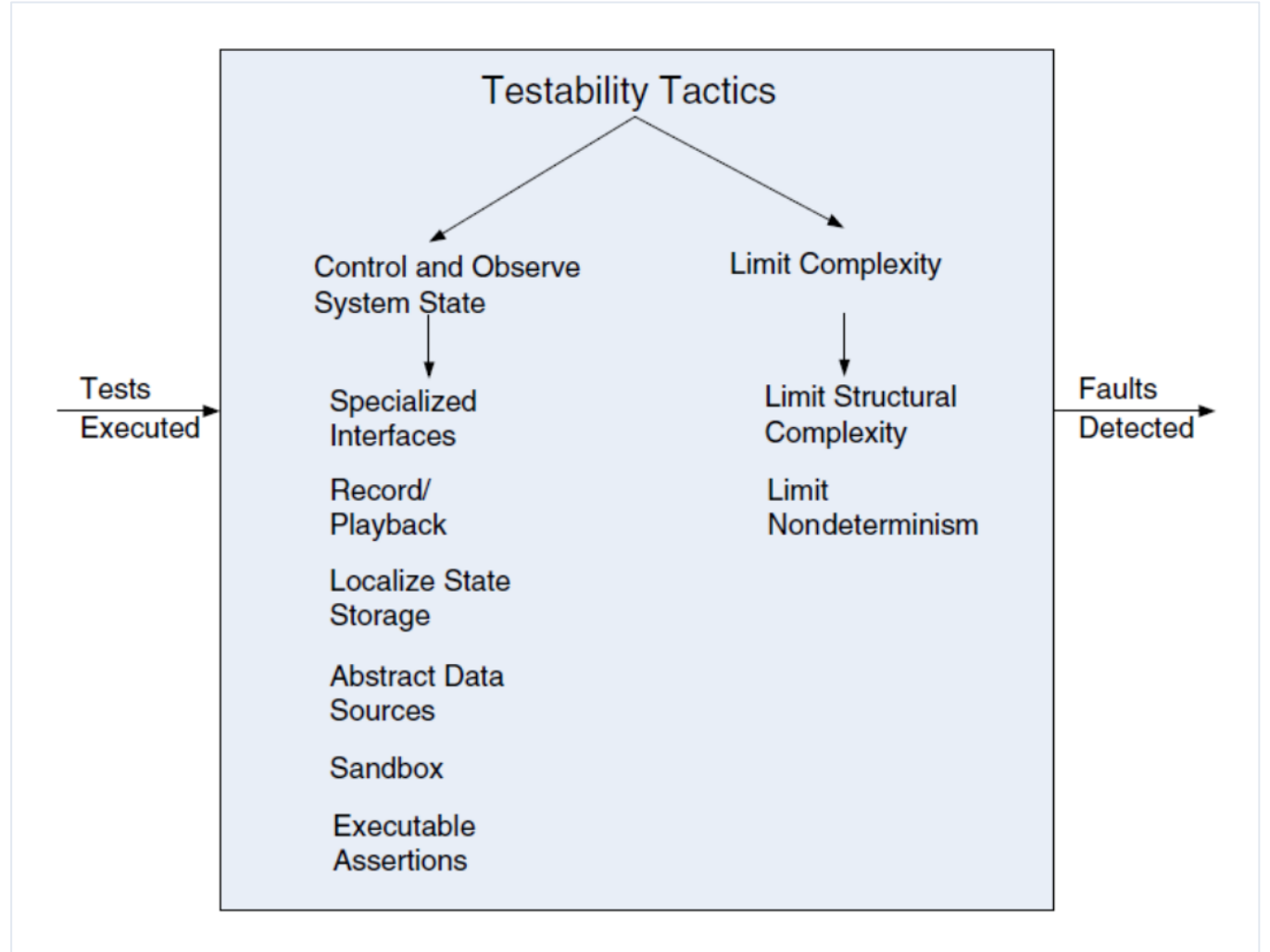
Testability 택틱 계층도

■ Control and Observe System State

- 제어(control)와 관찰(observation)은 Testability 의 핵심
- component 가 상태 정보를 유지하게 하고, tester 가 그 상태에 값을 넣거나, 필요할 때 그 정보에 접근할 수 있게 한다
- Specialized interfaces, Record/playback, Localize state storage, Abstract data sources, Sandbox, Executable assertions

■ Limit Complexity

- Limit structural complexity, Limit nondeterminism



Testability 택틱 계층도

Specialized Interfaces (전용 테스트 인터페이스)

■ 개념

- component 의 변수 값을 제어하거나 수집하기 위한 전용 테스트 인터페이스

■ 예

- set / get 메서드
- 객체의 전체 상태를 돌려주는 report 메서드
- 내부 상태를 지정한 상태로 되돌리는 reset 메서드
- 상세 출력, 여러 수준의 이벤트 logging, 자원 모니터링을 켜는 메서드

■ 주의

- 전용 테스트 인터페이스는 필요 시 제거할 수 있도록, 기능 요구사항용 인터페이스와 명확히 구분하거나 분리해서 둔다

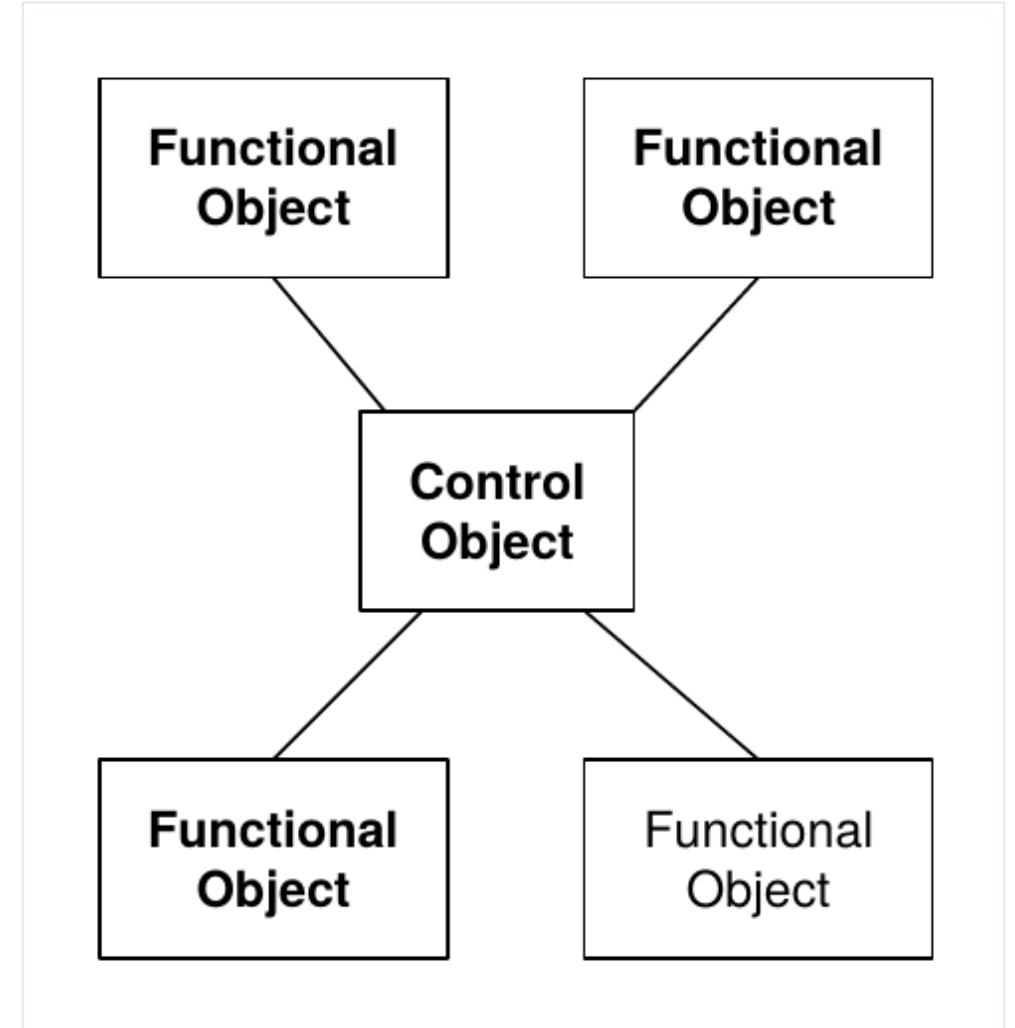
Record/Playback / Localize State Storage

■ Record/playback (기록/재생)

- 상태가 인터페이스를 지날 때 기록해 두면, 그 상태로 "시스템을 다시 재생"해 결함을 재현할 수 있다

■ Localize state storage (상태 저장소 국지화)

- 테스트를 위해 시스템·서브시스템·모듈을 임의의 상태에서 시작하려면 상태가 한 곳에 저장되어 있는 것이 가장 편리하다
- 상태 저장소를 "외부화"하는 편리한 방법은 state machine(또는 state machine 객체)을 현재 상태 추적·보고 메커니즘으로 쓰는 것
- 그림: 여러 Functional Object 의 상태를 하나의 Control Object 가 관리

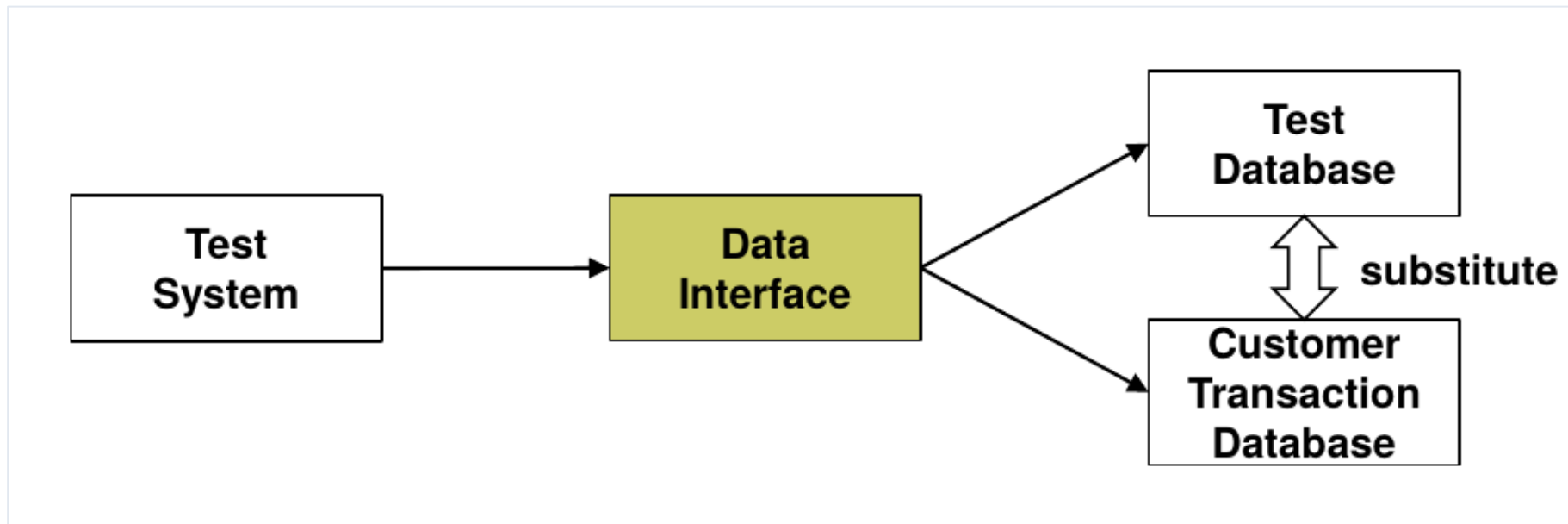


Control Object 에 상태를 모아 두는 구조

Abstract Data Sources (데이터 소스 추상화)

■ 개념

- 인터페이스를 추상화하면 테스트 데이터를 더 쉽게 바꿔 넣을 수 있다
- 예: 고객 transaction 데이터베이스가 있다면, 기능 코드를 바꾸지 않고도 테스트 시스템이 다른 테스트 DB 나 테스트 데이터 파일을 가리키도록 Architecture 를 설계할 수 있다



Data Interface 를 통해 Customer Transaction DB 를 Test DB 로 대체(substitute)

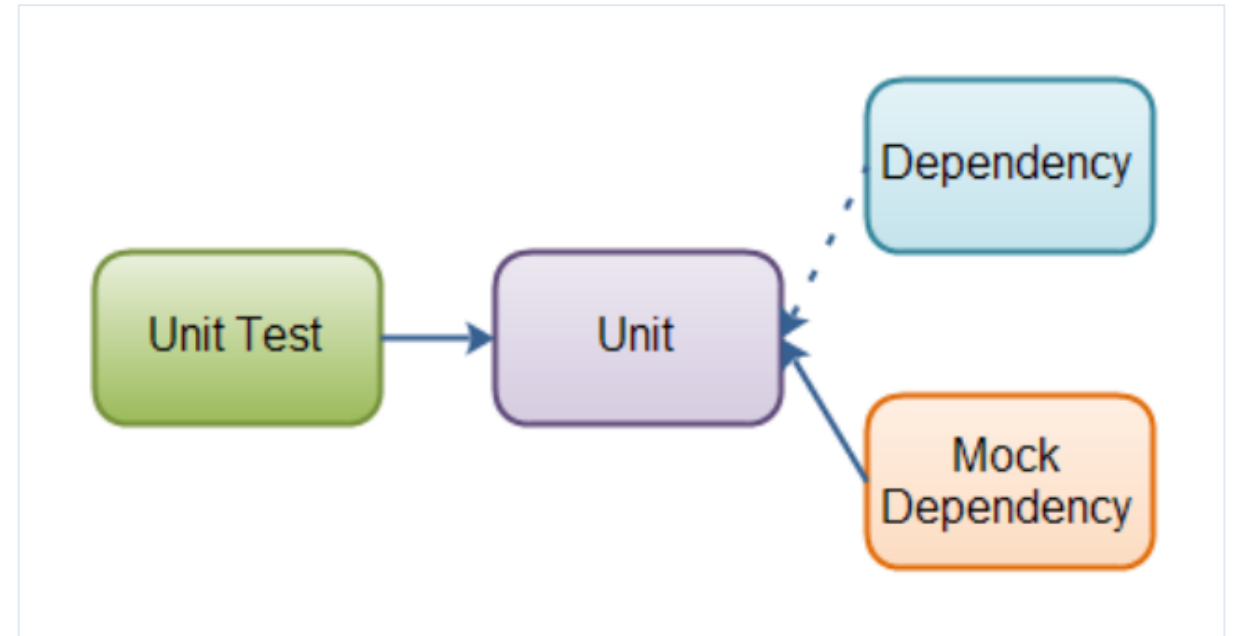
Sandbox (샌드박스)

■ 개념

- Sandbox 는 프로그램 실행이나 파일 실행이 애플리케이션·시스템·플랫폼에 영향을 주지 않도록 하는 격리된 테스트 환경

■ 자원의 가상화(virtualization)

- Sandbox 의 흔한 형태는 자원을 가상화하는 것
- 예: 시스템 시계(clock) 가상화
- 메모리, 배터리, 네트워크 등 다른 자원도 비슷하게 가상화할 수 있다
- Stub, Mock, Dependency Injection 은 단순하지만 효과적인 가상화 형태



Unit Test 에서 실제 Dependency 대신 Mock Dependency 를 주입

Executable Assertions (실행 가능한 단언문)

■ 개념

- Assertion 은 보통 손으로 작성해 원하는 위치에 배치하며, 데이터 값이 지정된 제약 조건을 만족하는지 검사한다
- 각 메서드의 사전 조건(pre-condition)·사후 조건(post-condition), 그리고 class 수준의 불변식(invariant)으로 표현할 수 있다
- 예: JUnit — Calculator.sum(10, 50) 의 결과가 60 인지 assertEquals 로 검사

```
public class Calculator {  
    public double sum(double a,  
double b){  
        return a + b;  
    }  
}
```

```
@Test  
public void testSum(){  
    Calculator c = new Calculator();  
    double result = c.sum(10, 50);  
    assertEquals(60, result, 0);  
}
```

JUnit assertion 예제

Limit Complexity (복잡도 제한)

■ 왜 복잡도를 제한하는가

- 복잡한 소프트웨어는 테스트하기 더 어렵다
- 테스트는 failure 를 일으킨 fault 를 찾아 제거하는 것
 - 따라서 동작을 반복 가능(repeatable)하게 만드는 것이 관심사

■ 이 범주의 두 택틱

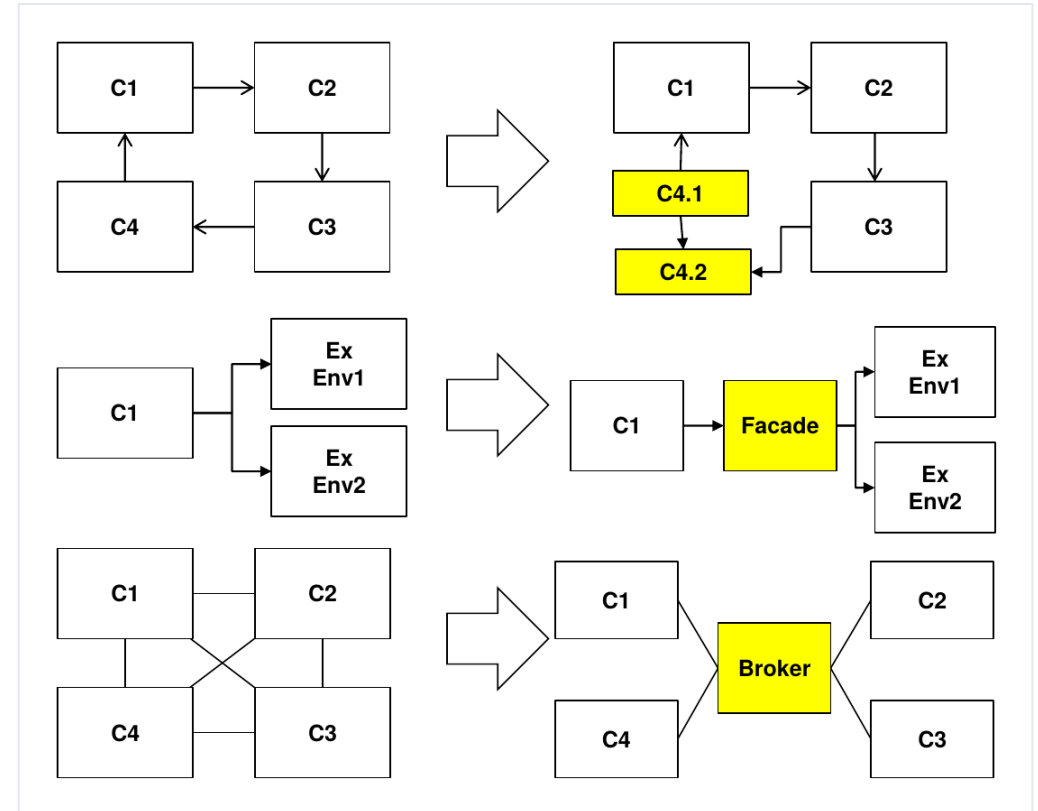
- Limit structural complexity (구조적 복잡도 제한)
- Limit nondeterminism (비결정성 제한)

핵심 단순한 시스템일수록 같은 입력에 같은 결과를 내므로 결함을 재현하고 찾기 쉽다

Limit Structural Complexity (1) — 의존성 줄이기

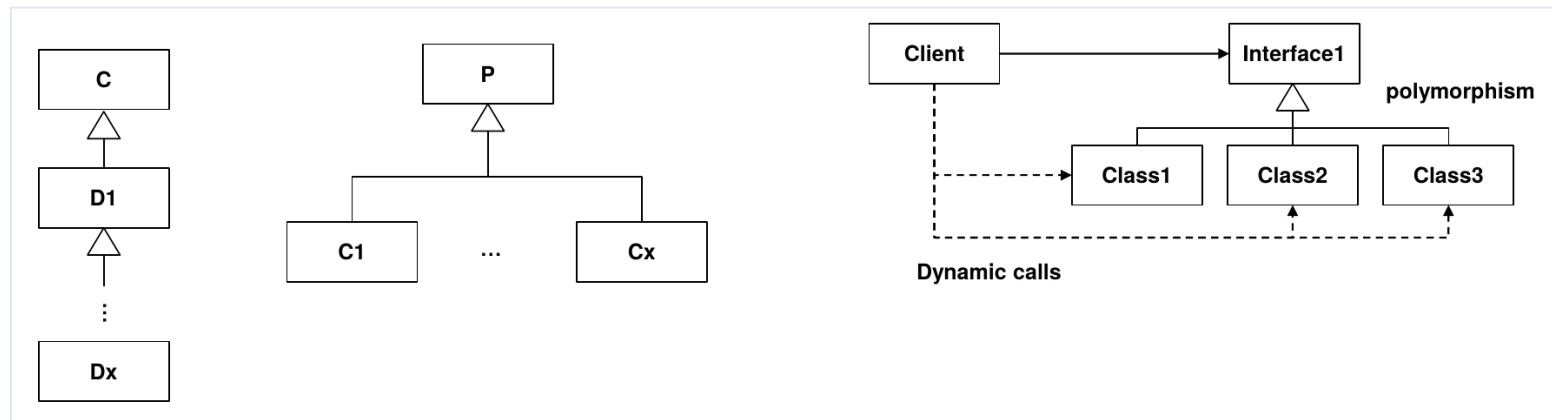
■ 세 가지 방법

- component 사이의 순환 의존성(cyclic dependency)을 피하거나 해소한다
 - C4 를 C4.1, C4.2 로 나누어 순환을 끊는다
- 외부 환경에 대한 의존성을 격리하고 캡슐화한다
 - Facade 를 두어 외부 환경(Ex Env)에 대한 의존을 한 곳에 모은다
- component 사이의 의존성을 전반적으로 줄인다
 - Broker 를 두어 component 끼리 직접 연결되지 않게 한다



순환 의존 해소 / Facade / Broker 를 통한 의존성 감소

Limit Structural Complexity (2) — 상속과 다형성



상속 깊이·자식 수 제한 / 다형성과 동적 호출 제한

■ 상속 트리 제한

- 상속 트리의 깊이($C \rightarrow D1 \rightarrow \dots \rightarrow Dx$)와 한 class 의 자식 수($P \rightarrow C1 \dots Cx$)를 제한한다

■ 다형성과 동적 호출 제한

- Client** 가 **Interface1** 을 통해 **Class1~3** 을 다형적으로 호출하면 실제 실행 경로를 예측하기 어려워지므로 제한한다

■ Modifiability 태틱도 도움이 된다

- 높은 응집도(high cohesion), 낮은 결합도(loose coupling), 관심사의 분리(separation of concerns)

Limit Nondeterminism (비결정성 제한)

■ 개념

- 제약 없는 병렬성(unconstrained parallelism) 같은 비결정성의 원천을 모두 찾아내어 가능한 한 제거한다
- 예: seed 값으로 무작위성(randomness)을 제어
- 예: 동시 실행되는 부분을 최소화

■ 피할 수 없는 비결정성

- 예측 불가능한 이벤트에 반응하는 multithread 시스템처럼 비결정성을 없앨 수 없는 경우가 있다
- 이런 시스템에는 Record/playback 같은 다른 택틱을 사용한다

Testability Tactics 정리

■ Control and Observe System State

- 시스템 상태를 제어하고 관찰하는 것이 Testability Tactics의 주요 부류
- Specialized Interfaces, Record/Playback, Localize State Storage, Abstract Data Sources, Sandbox, Executable Assertions

■ Limit Complexity

- 복잡한 시스템은 테스트하기 어렵다
- 따라서 시스템을 단순하게 유지하는 것이 Testability를 지원하는 또 다른 Tactics 부류
- Limit Structural Complexity, Limit Nondeterminism

3.11

Usability Tactics

사용자가 원하는 일을 쉽고 효율적으로 끝내도록 돕는 تاک

- Usability 의 정의와 관심사
- General / Concrete Usability Scenario
- Support User Initiative (Cancel, Undo, Pause/Resume, Aggregate)
- Support System Initiative (Task / User / System Model)
- Modifiability 와의 연결

Usability(사용성)란 무엇인가

■ Usability 의 관심사

- 사용자가 원하는 작업을 얼마나 쉽게 끝낼 수 있는가
- 시스템이 사용자에게 어떤 종류의 지원을 제공하는가

■ 관련 이슈와 예

- 시스템 기능 학습하기 — 예: help 기능
- 시스템을 효율적으로 사용하기 — 예: suspend/resume 기능
- 오류의 영향 최소화하기 — 예: cancel 기능
- 사용자 요구에 맞게 시스템 적응시키기 — 예: 자동 입력
- 자신감과 만족도 높이기 — 예: progress bar

핵심 Usability 는 '얼마나 쉽게 일을 끝내는가'와 '시스템이 얼마나 도와주는가'의 문제이다

General Usability Scenario

항목	내용
Source	End user(최종 사용자)
Stimulus	사용자가 원하는 것: 시스템 기능 학습 / 효율적 사용 / 오류 영향 최소화 / 시스템 적응 / 편안함 느끼기
Artifact	시스템 전체 또는 시스템의 특정 부분
Environment	Runtime 또는 configure time(설정 시점)
Response	시스템이 사용자에게 필요한 기능을 제공하거나, 사용자의 요구를 미리 예측한다
Response Measure	작업 시간, 오류 수, 완료한 작업 수, 사용자 만족도, 사용자 지식 향상, 전체 연산 대비 성공 연산 비율, 오류 발생 시 잃어버린 시간/데이터 양

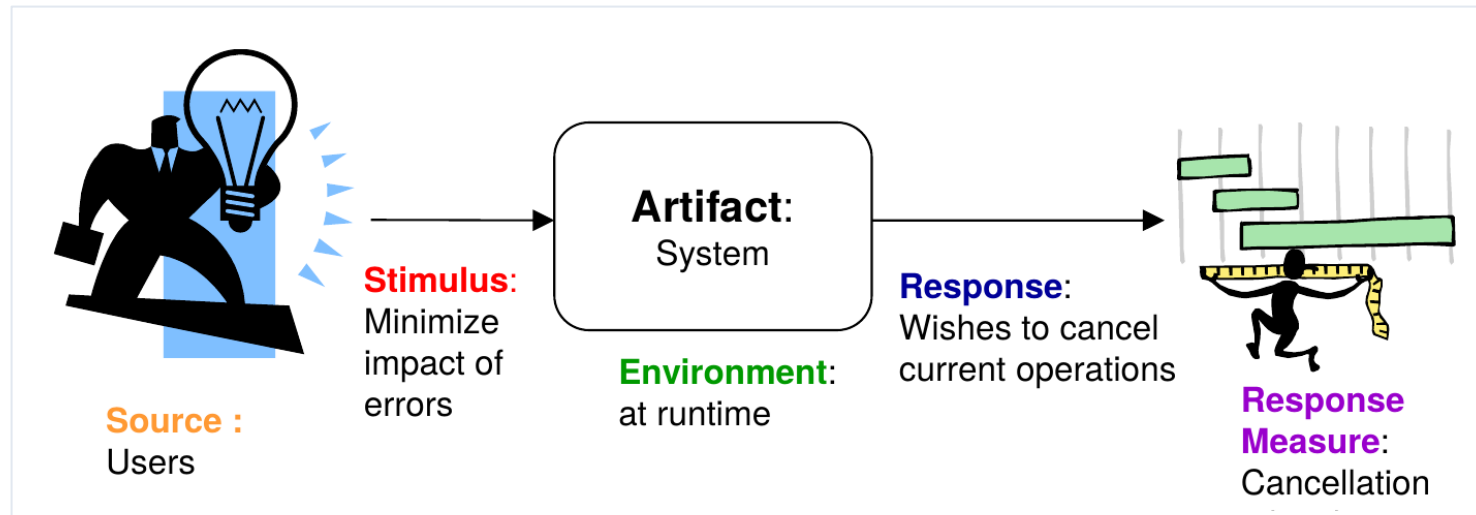
Concrete Usability Scenario 예

■ 시나리오

- "오류의 영향을 최소화하려는 사용자가 runtime 에 시스템 연산을 취소하고자 한다. 취소는 1초 이내에 이루어진다"

■ 6 요소로 분해

- Source: Users / Stimulus: 오류 영향 최소화 / Artifact: System
- Environment: runtime / Response: 현재 연산 취소 / Response Measure: 취소가 1초 이내에 완료



Concrete Usability Scenario 의 6 요소

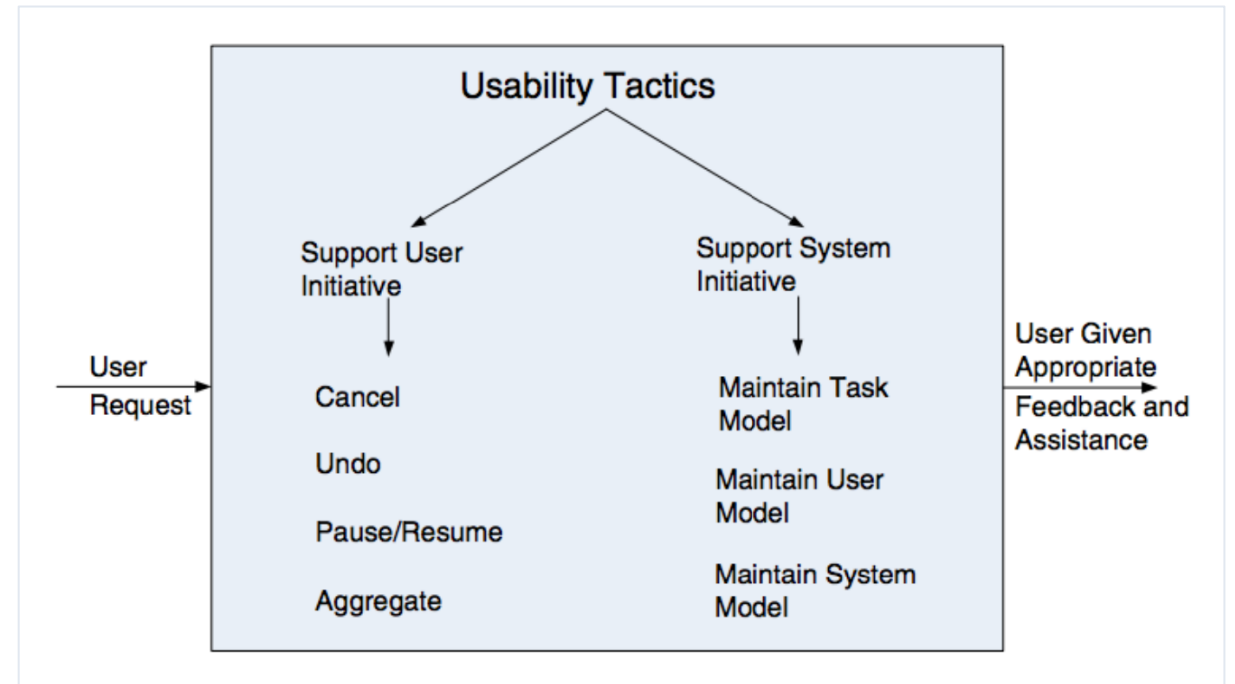
Usability Tactics 분류

■ 두 갈래의 택틱

- Support User Initiative: 사용자가 주도적으로 행동하도록 지원
 - Cancel, Undo, Pause/Resume, Aggregate
- Support System Initiative: 시스템이 주도적으로 사용자를 돕도록 지원
 - Maintain Task Model, Maintain User Model, Maintain System Model

■ 입력과 결과

- User Request → 택틱 적용 → 사용자에게 적절한 feedback 과 assistance 제공



Usability Tactics 의 분류

Support User Initiative — 개요

■ 기본 아이디어

- 사용자가 적절한 응답(행동)을 할 수 있게 허용함으로써 usability 를 높인다

■ 관련택틱

- 사용자의 오류 수정을 지원하는택틱
 - Cancel
 - Undo
- 사용자가 더 효율적으로 작업하도록 지원하는택틱
 - Pause/Resume
 - Aggregate

오류 수정 지원: Cancel 과 Undo

■ Cancel

- 사용자가 cancel 명령을 내리면 시스템은 그 명령을 항상 듣고(listen) 있어야 한다
- 취소 대상 명령을 종료해야 한다
- 취소된 명령이 사용하던 자원을 모두 해제해야 한다
- 취소된 명령과 협력 중이던 component 들에게 취소 사실을 알려야 한다

■ Undo

- 이전 상태로 되돌릴 수 있도록, 시스템 상태에 관한 충분한 정보를 유지해야 한다

핵심 Cancel 은 '지금 하던 일을 멈추는 것', Undo 는 '이미 한 일을 되돌리는 것'이다

효율적 사용 지원: Pause/Resume 과 Aggregate

■ Pause/Resume

- 사용자가 오래 걸리는 작업(예: 서버에서 큰 파일 다운로드)을 시작했을 때
 - 작업을 잠시 멈추고 다시 이어갈 수 있는 기능을 제공하면 유용하다

■ Aggregate

- 사용자가 반복적인 작업을 하거나, 많은 객체에 같은 작업을 적용할 때
 - 하위 수준 객체들을 하나의 그룹으로 묶어(aggregate) 한 번에 처리하는 기능을 제공하면 유용하다

Support System Initiative — 개요

■ 시스템이 사용자를 돕기 위해 필요한 것

- 사용자에 대한 model
- 사용자가 수행 중인 task
- 시스템 자체의 상태

■ 택틱의 원리

- 이런 model 들을 유지하고 활용하여 시스템의 동작 또는 사용자의 의도를 예측한다
- Maintain task model
- Maintain user model
- Maintain system model

핵심 시스템이 사용자·작업·자기 자신에 대한 명시적 model 을 갖고 있어야 먼저 도움을 줄 수 있다

Task / User / System Model 유지

■ Maintain task model

- 사용자의 context(지금 무엇을 하려는지)를 파악해 도움을 준다
- 예: 문장 첫 글자의 소문자를 자동으로 대문자로 고쳐 준다

■ Maintain user model

- 사용자의 시스템 지식과 행동을 명시적으로 표현한다
- 예: 마우스 선택 속도를 조절해 문서 전체가 한꺼번에 선택되지 않게 한다

■ Maintain system model

- 시스템의 예상 동작을 파악해 사용자에게 적절한 feedback 을 준다
- 예: 현재 작업이 끝나기까지 남은 시간을 보여 주는 progress bar

Modifiability 와의 연결

■ 왜 Modifiability 인가

- 시스템의 usability 를 높이려면 user interface 를 빠르고 효율적으로 바꿀 수 있어야 한다
- Modifiability 택틱이 이 목표를 뒷받침한다

■ 도움이 되는 Modifiability 택틱

- Increase semantic coherence, Encapsulate, Co-locate related responsibilities
 - UI 책임을 한 곳에 모아 localize 한다
- Restrict dependencies
 - UI 가 바뀔 때 다른 소프트웨어로 퍼지는 ripple effect 를 최소화한다
- Defer binding
 - 코드를 다시 짜지 않고도 중요한 UI 선택을 바꿀 수 있게 한다

Usability Tactics 요약

■ Support User Initiative

- Cancel, Undo, Pause/Resume, Aggregate
- 사용자가 시스템을 효율적으로 사용하고, 오류의 영향을 최소화하도록 돕는다

■ Support System Initiative

- Maintain task model, Maintain user model, Maintain system model
- 사용자·시스템·작업에 대한 명시적 model 을 유지해 사용자나 시스템의 응답을 예측한다

핵심 Usability 는 사용자 주도 지원과 시스템 주도 지원, 두 종류의 태틱으로 달성한다

3.12

Tactics and Architectural Patterns

택틱과 패턴의 관계 — 패턴은 택틱의 묶음이며, 택틱으로 패턴을 보강한다

- Tactics 와 Architectural Pattern 의 관계
- Layered Style 속의 택틱
- Active Object Pattern 속의 택틱
- 패턴별 대응 택틱 표
- 택틱으로 Broker Pattern 보강하기
- 설계에서 택틱 적용하기 (Ping/Echo 예제)

Tactics 와 Architectural Patterns

■ Architecture Patterns

- Architecture 의 한 부류(architecture style)
- 실무에서 반복적으로 발견되는 설계 결정(design decision)의 묶음(package)

■ Tactics

- 설계의 기초 building block
- 택틱을 조합해 architectural pattern 이나 style 이 만들어진다

핵심 택틱과 패턴 모두 검증된 좋은 설계 해법을 재사용할 수 있게 포착한 것이다

Layered Style 속의 택틱

■ Layered style 은 여러 택틱의 결합체(amalgam)로 볼 수 있다

■ Increase semantic coherence

- 한 layer 의 책임들은 다른 layer 에 지나치게 의존하지 않고 함께 동작해야 한다
- 의미적으로 응집된(semantic coherence) 책임들을 골라 배치함으로써 달성한다

■ Restrict dependencies

- Layer 는 순서를 정의하고, 바로 아래 인접 layer 의 서비스만 사용하도록 허용한다
- 이 제한이 layer 간 의존성을 크게 줄여, layer 를 교체할 때 부작용을 제한하기 훨씬 쉽다

Active Object Pattern 이란

■ 정의 [SSR+00]

- Method 의 실행(execution)과 호출(invocation)을 분리 (decouple)하는 design pattern
- 동시성(concurrency)을 높이고, 자기 thread 를 가진 객체에 대한 동기화된 접근을 단순하게 만든다

■ 6개의 구성 요소

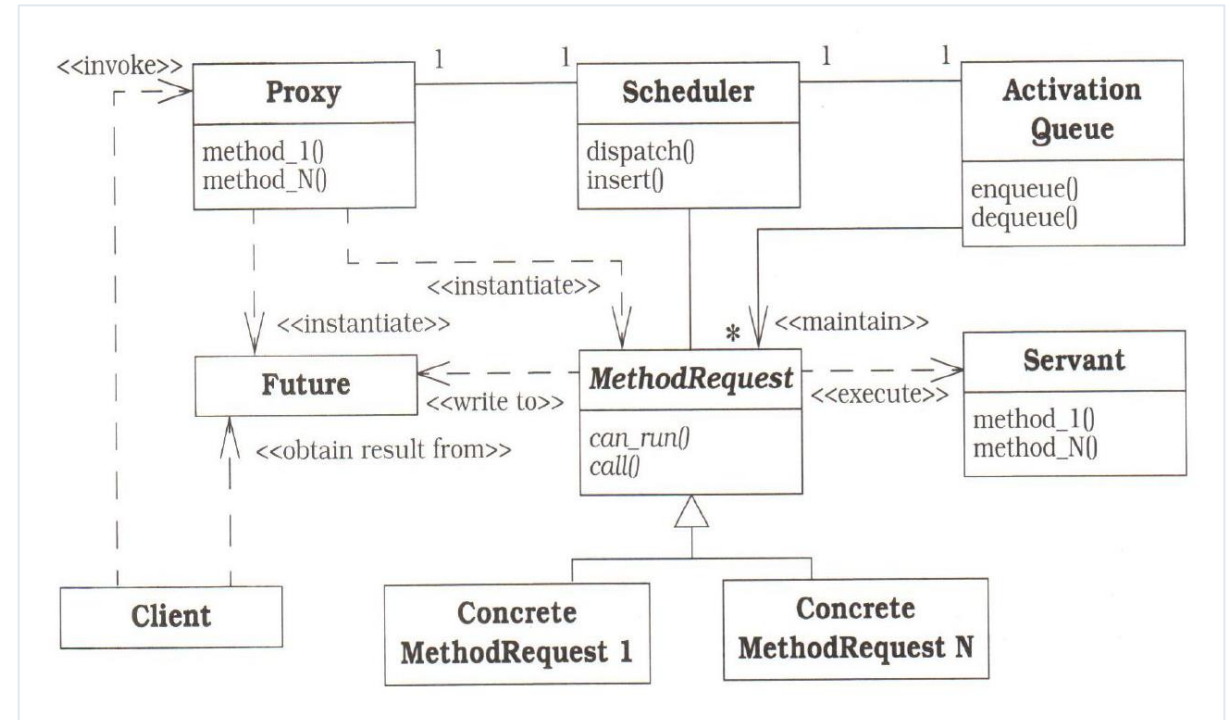
- Proxy: client 가 active object 의 public method 를 호출할 수 있는 interface 제공
- Method Request: active object 의 method 를 실행하기 위한 interface 정의
- Activation Queue: 대기 중인 method request 의 buffer 유지
- Scheduler: 다음에 실행할 method request 를 결정
- Servant: active object 로 모델링된 동작과 상태를 정의
- Future: client 가 method 호출 결과를 나중에 얻을 수 있게 함

핵심 출처: Schmidt, Stal, Rohnert, Buschmann, POSA Vol.2 (Wiley, 2000)

Active Object Pattern — 구조

■ 클래스 구조 읽기

- Client 는 Proxy 를 invoke 하고, Proxy 는 MethodRequest 와 Future 를 instantiate 한다
- Scheduler 는 Activation Queue 에 request 를 insert 하고 dispatch 한다
- Scheduler 가 MethodRequest 를 execute 하면 Servant 의 method 가 실행된다
- 결과는 Future 에 write 되고, Client 는 Future 에서 결과를 obtain 한다

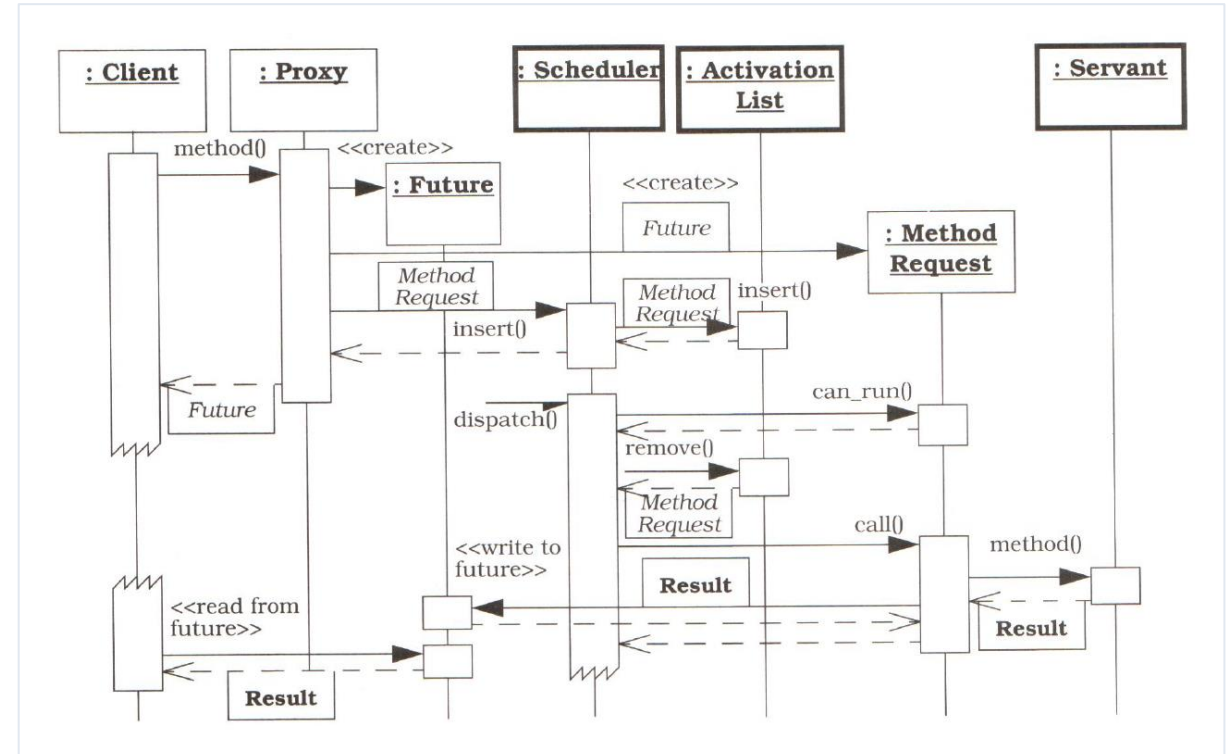


Active Object Pattern 의 구조(클래스 다이어그램)

Active Object Pattern — 동작

■ Sequence 읽기

- Client 가 Proxy 의 method() 를 호출하면 Proxy 가 Future 와 Method Request 를 만든다
- Proxy 는 Method Request 를 Scheduler 에 insert 하고, Client 에게 Future 를 즉시 돌려준다
- Scheduler 는 dispatch 시 can_run() 으로 실행 가능한 request 를 골라 queue 에서 remove 한다
- call() 로 Servant 의 method() 가 실행되고, 결과는 Future 에 write 된다
- Client 는 나중에 Future 에서 Result 를 read 한다



Active Object Pattern 의 동작(시퀀스 다이어그램)

Active Object Pattern 속의 تاک

■ 주 목적

- Performance 향상을 위한 "Introduce concurrency" تاک을 구현하는 것
- 그러나 이 패턴에는 다른 تاک들도 함께 들어 있다

■ 함께 포함된 تاک

- Encapsulate (Modifiability): 각 요소가 책임의 구현 세부를 interface 뒤에 숨긴다
- Use an intermediary (Modifiability): Proxy 가 중개자 역할을 하여 method 호출의 변경을 완충한다
- Defer binding time (Modifiability): Client 와 Proxy 의 binding 시점을 열어 둔다
- Schedule resources (Performance): Scheduler 가 특정 scheduling policy 를 구현한다

핵심 하나의 패턴 = 여러 Quality Attribute 의 تاک들이 결합된 묶음

Architectural Patterns 와 대응 Tactics

■ 표 읽기 (Modifiability 태틱 기준)

- Increase Cohesion: Increase Semantic Coherence, Abstract Common Services
- Reduce Coupling: Encapsulate, Use a Wrapper, Restrict Comm. Paths, Use an Intermediary, Raise the Abstraction Level
- Defer Binding Time: Use Runtime Registration, Use Startup-Time Binding, Use Runtime Binding
- Layered, Broker, Blackboard 등 각 패턴이 어떤 태틱을 포함하는지 X 로 표시

Pattern	Modifiability								
	Increase Cohesion		Reduce Coupling				Defer Binding Time		
	Increase Semantic Coherence	Abstract Common Services	Encapsulate	Use a Wrapper	Restrict Comm. Paths	Use an Intermediary	Raise the Abstraction Level	Use Runtime Registration	Use Startup-Time Binding
Layered	X	X	X		X	X	X		
Pipes and Filters	X		X		X	X			X
Blackboard	X	X			X	X	X	X	
Broker	X	X	X		X	X	X	X	
Model View Controller	X		X			X			X
Presentation Abstraction Control	X		X			X	X		
Microkernel	X	X	X		X	X			
Reflection	X		X						

참고. L. Bass et al., Software Architecture in Practice, Third Edition, Ad

패턴별 Modifiability 태틱 대응표 (출처: Bass et al., SAiP 3rd ed.)

택틱으로 패턴 보강하기 — Broker Pattern

■ 패턴 선택 이후에도 결정은 남는다

- Architectural style/pattern 은 어떤 quality attribute 는 촉진하고, 다른 것은 저해한다
- 패턴(또는 style)을 골랐더라도 여전히 많은 설계 결정이 남아 있다

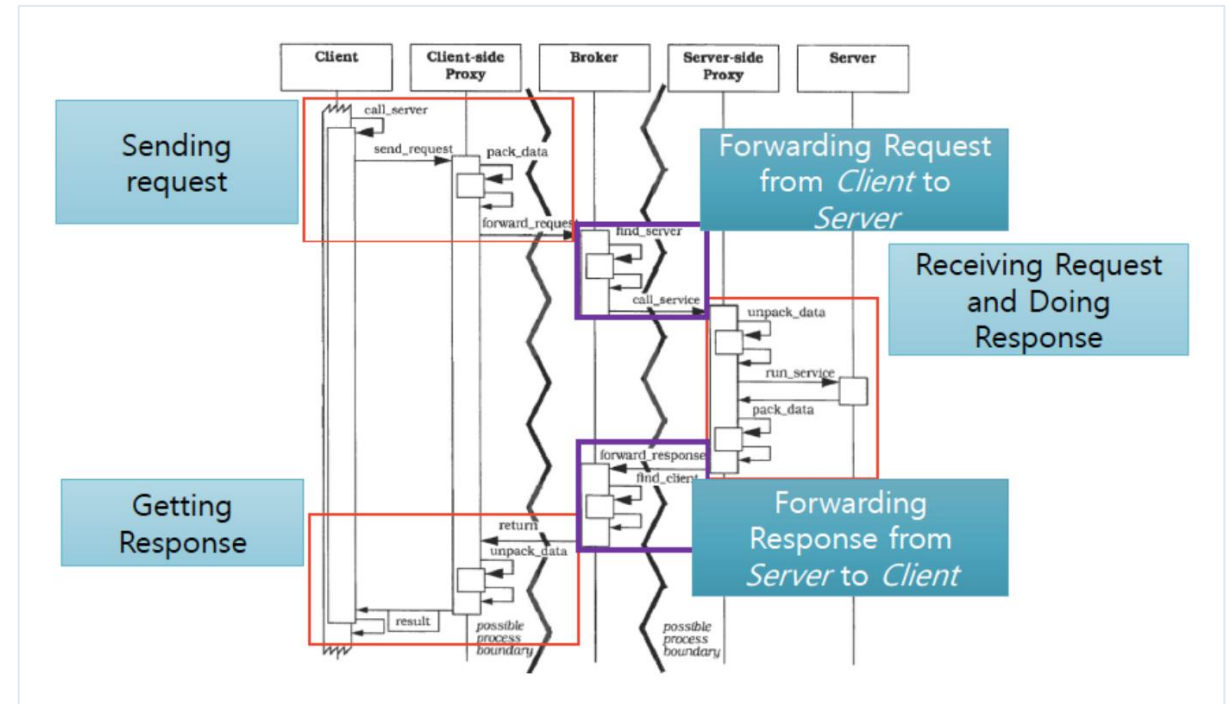
■ Broker Pattern 복습

- 서버에 정보를 요청하는 client 는 서버의 위치나 API 를 알 필요가 없다
- Client 는 단지 broker 에 접촉하면 된다 (보통 client-side proxy 를 통해)

Broker Pattern 의 동작(Dynamics)

■ 요청과 응답의 흐름

- Sending request: Client 가 Client-side Proxy 를 통해 요청을 보낸다 (pack_data, send_request)
- Forwarding request: Broker 가 서버를 찾아 (find_server) Server-side Proxy 로 요청을 전달한다
- Receiving & doing: 서버가 데이터를 풀어 (unpack_data) 서비스를 실행(run_service)한다
- Forwarding & getting response: 응답이 Broker → Client-side Proxy → Client 로 되돌아온다



Broker Pattern 의 요청/응답 시퀀스

Broker Pattern 의 약점

■ Availability

- Broker 는 single point of failure 이다
- 서버, broker, 심지어 client 의 liveness 까지 모니터링해야 하고, 복구(repair) 메커니즘도 제공해야 한다

■ Performance

- Client(요청자)와 Server(제공자) 사이의 여러 단계 간접 참조(indirection)가 overhead, 즉 latency 를 더한다
- 보안 등의 이유로 client-server 직접 통신을 허용하지 않으면, broker 가 잠재적 성능 병목이 된다

택틱으로 Broker Pattern 개선하기

■ Maintain multiple copies

- Broker 를 여러 개 두어 performance 와 availability 를 함께 개선한다

■ Load balancing (Schedule resources 택틱의 응용)

- 한 broker 는 과부하인데 다른 broker 는 노는 상황을 막는다

■ Heartbeat, Exception detection, Ping/Echo (fault detection)

- 복제된 broker 들이 자신이 서비스 불능일 때 client 와 서로에게 알릴 수 있는 수단을 제공한다

핵심 패턴의 약점을 알면, 그 약점을 겨냥한 택틱을 골라 패턴을 우리 문제에 맞게 구체화할 수 있다

설계에서 태틱 적용하기

■ 태틱의 성격

- Tactic 은 하나의 quality attribute response 를 다루기 위한 design primitive 이다
- 모든 태틱에는 side effect 와 tradeoff 가 있다

■ 적용 전에 할 일

- Architect 는 태틱을 설계에 적용하기 전에 그 태틱이 가져올 design implication(side effect 와 tradeoff) 을 고려해야 한다

핵심 태틱은 공짜가 아니다 — 하나를 고치면 다른 품질 속성에 영향이 간다

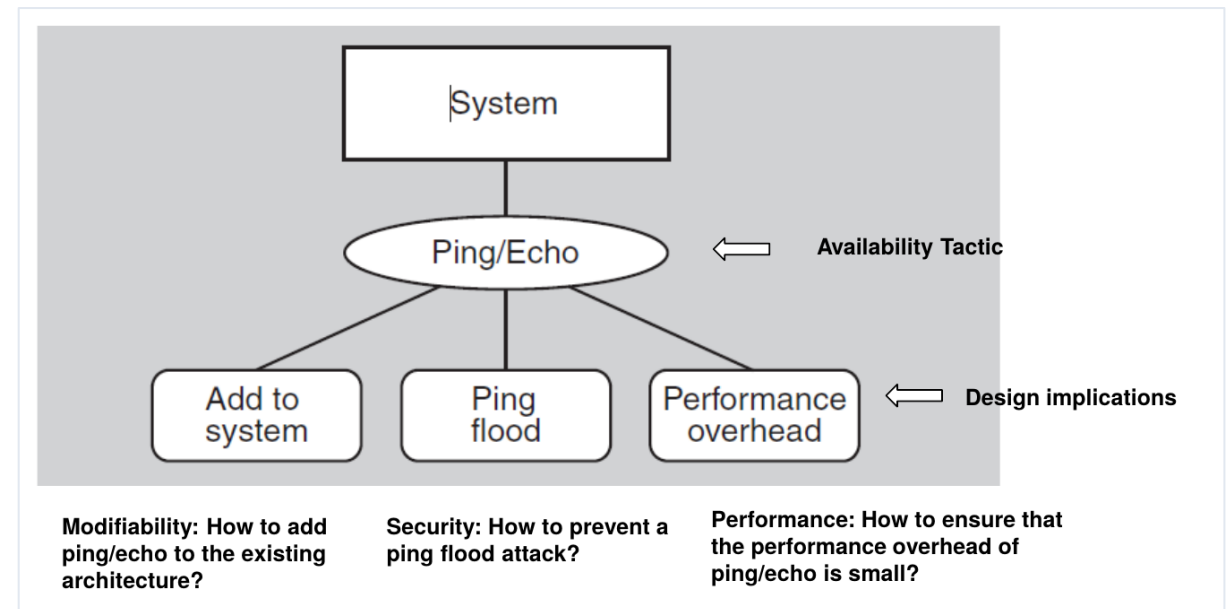
택틱 적용 예 (1) — Ping/Echo 도입

■ 상황

- Component 의 fault 를 탐지해야 하는 시스템을 생각해 보자
- Availability 택틱으로 Ping/Echo 를 선택한다

■ Ping/Echo 의 design implications

- Modifiability: 기존 Architecture 에 ping/echo 를 어떻게 추가할 것인가?
- Security: ping flood 공격을 어떻게 막을 것인가?
- Performance: ping/echo 의 성능 overhead 를 어떻게 작게 유지할 것인가?



Availability 택틱(Ping/Echo)과 그 design implications

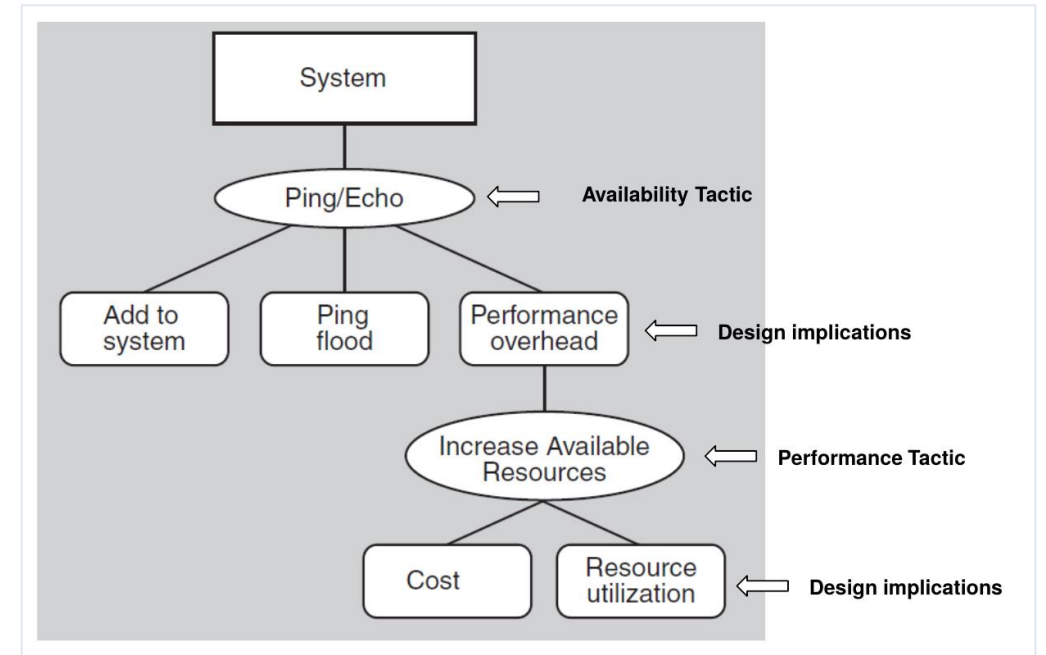
택틱 적용 예 (2) — Performance 부작용 대응

■ Architect 의 판단

- Performance tradeoff(ping/echo 추가로 인한 overhead)가 가장 심각하다고 판단했다고 하자

■ 대응 택틱: Increase available resources

- Performance 부작용을 해결하기 위한 택틱
- 이 택틱의 design implications:
 - Cost: 자원을 늘리면 비용이 더 든다
 - Performance: 늘어난 자원을 어떻게 효율적으로 활용할 것인가?



Performance 택틱을 덧붙인 결과와 새로운 design implications

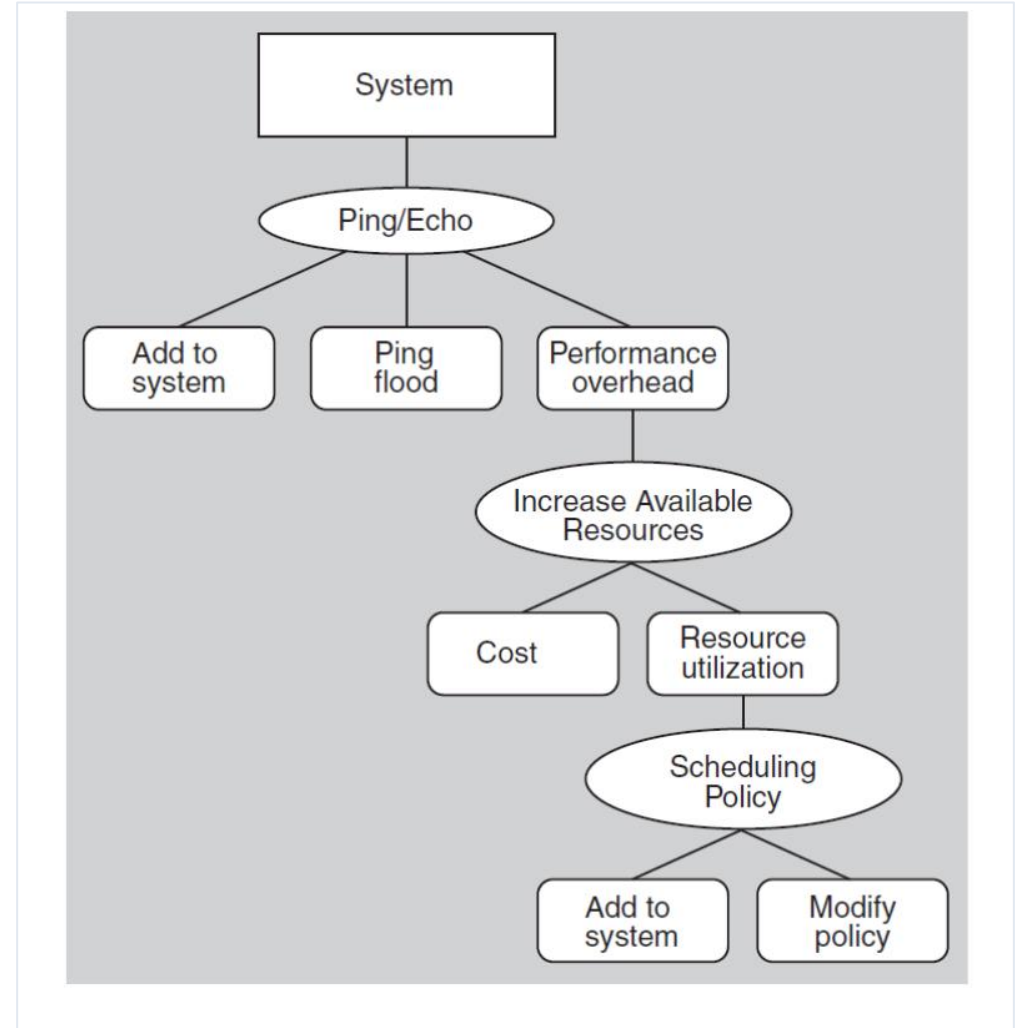
택틱 적용 예 (3) — 자원 활용 문제와 체스 비유

■ 대응 택틱: Scheduling policy

- 자원 활용(resource utilization) 문제를 더 다루기 위해 자원을 효율적으로 쓰는 택틱을 고른다
- 이 택틱의 design implications:
 - Modifiability: scheduling policy 를 기존 Architecture 에 어떻게 추가할 것인가?
 - Modifiability: 나중에 scheduling policy 를 어떻게 바꿀 것인가?

■ 체스 비유

- 택틱을 연달아 적용하는 것은 게임 공간을 옮겨 다니는 것, 마치 체스와 같다
- 다음 수의 결과가 중요하며, 몇 수 앞을 내다보면 도움이 된다



연속 적용된 택틱과 design implications 의 트리

Tactics and Architectural Patterns 요약

■ 핵심 정리

- Architect 는 택틱을 사용해 특정 quality attribute 를 실현한다
- Architectural tactic 은 quality attribute 를 만족시키기 위해 Architect 가 선택하는 설계 옵션이다
- Architectural pattern 은 택틱들의 묶음(package)이다
- 패턴에 택틱을 덧붙여(augment) 우리 문제에 더 잘 맞도록 구체화할 수 있다

핵심 패턴 = 택틱의 묶음, 택틱 = 패턴을 우리 문제에 맞게 다듬는 도구

4

Architecture Design (ADD)

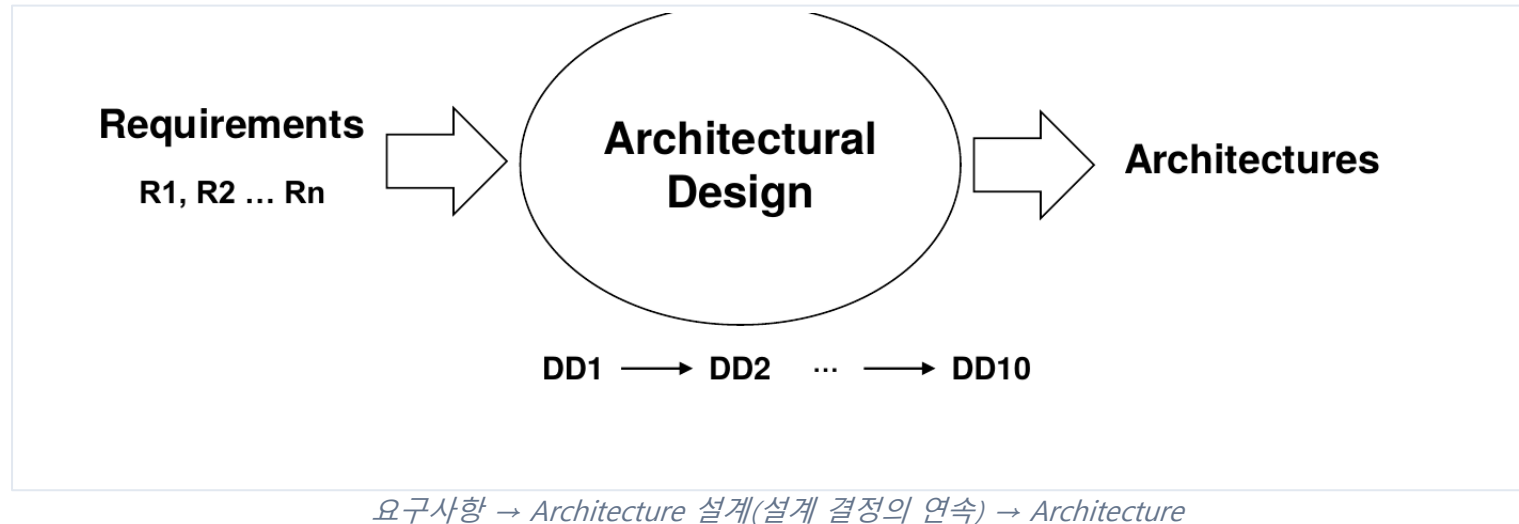
Tactic 과 Pattern 을 활용해 요구사항을 만족하는 Architecture 를 체계적으로 설계하는 방법

- Architecture 설계(Architectural Design)란
- Architectural Drivers
- Design Concepts
- Attribute-Driven Design (ADD) 3.0
- Case Study: FCAPS System

Architecture 설계(Architectural Design)란?

■ 정의

- Architectural Design(Architecture 설계)은 주어진 요구사항을 만족시키는 일련의 Design Decision(설계 결정)을 내려 Architecture 를 만들어 가는 활동이다
- 요구사항 R1, R2 ... Rn 이 입력으로 들어가고, 설계 결정 DD1 → DD2 → ... → DD10 이 차례로 쌓여 Architecture 가 만들어진다

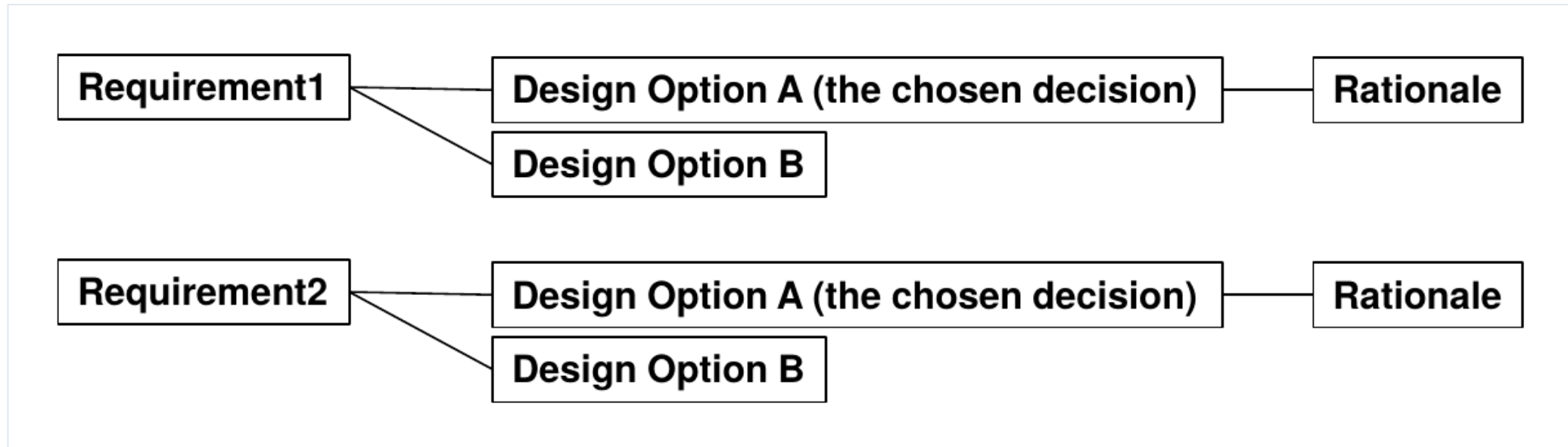


핵심 Architecture 는 한 번에 그려지는 것이 아니라, 요구사항에 대한 설계 결정을 차례로 내린 결과물이다

Architecture 설계는 체계적으로 수행해야 한다

■ 체계적인 설계의 세 가지 원칙

- 설계 결정은 요구사항을 기준으로 내린다
- 결정을 내리기 전에 가능한 Design Option(설계 대안)을 찾아보고 평가한다
- 설계 결정은 기록하고, 왜 그렇게 결정했는지(Rationale)를 함께 남긴다



요구사항마다 대안 A/B 를 비교해 하나를 선택하고 근거(Rationale)를 기록한다

Architectural Drivers 란?

■ 정의

- Architectural Driver 는 소프트웨어 Architecture 의 모양을 결정짓는 요구사항이다

■ 핵심 Driver 세 가지

- Primary Functionality(주요 기능): 비즈니스 목표 달성에 결정적인 기능
- Constraints(제약사항): 설계자 대신 이미 내려져 있는 설계 결정
- Quality Attributes(품질 속성): 기능 외에 시스템이 갖추어야 할 특성

■ 그 밖의 Driver

- Design Purpose(설계 목적): 이 설계로 달성하려는 목적
- Architectural Concerns(Architecture 관심사): 추가로 고려해야 할 측면

핵심 Architecture 를 설계하기 전에 다섯 가지 Driver 를 먼저 정리한다

Primary Functionality (주요 기능)

■ 주요 기능의 판단 기준

- 비즈니스 목표를 직접 지원한다
- 기술적 난이도가 높다
- 많은 Architecture 요소 사이의 상호작용이 필요하다

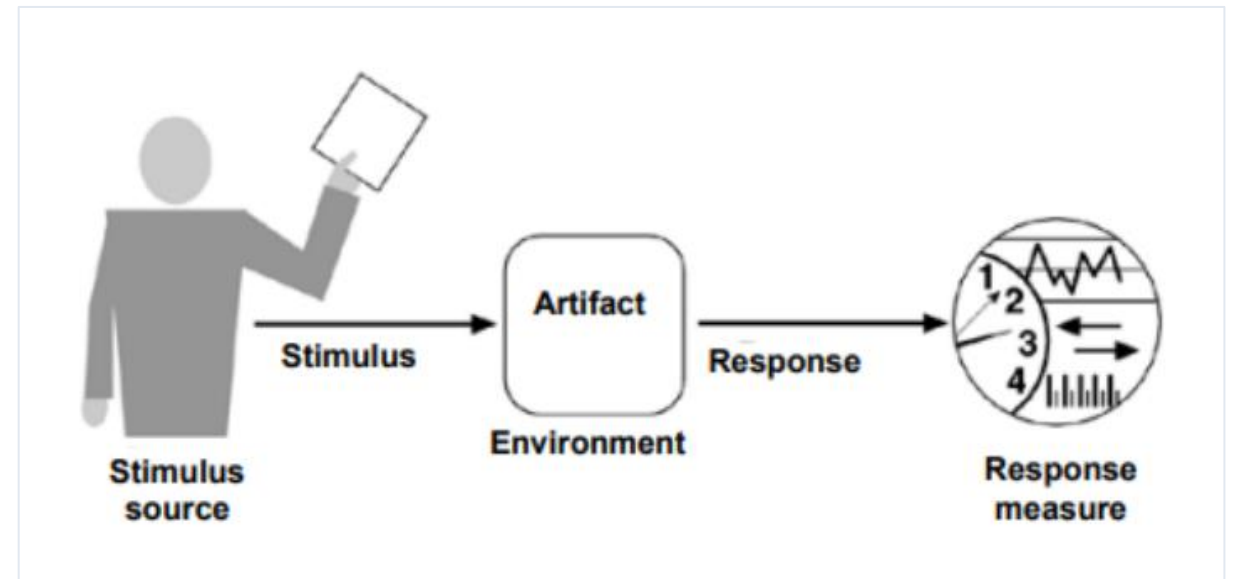
■ 경험 법칙

- 전체 use case 또는 user story 의 약 10% 정도가 primary 에 해당한다
- 나머지 기능을 어느 module 에 배치할지는, 주요 기능의 배치 결정을 바탕으로 정한다
 - 즉, 주요 기능이 Architecture 의 뼈대를 잡고 나머지 기능은 그 뼈대 위에 얹는다

Quality Attributes (품질 속성)

■ Architecture 를 가장 크게 좌우하는 Driver

- Driver 중에서 Quality Attribute(품질 속성)가 Architecture 의 형태에 가장 큰 영향을 준다
- 사용자와 개발자가 관심을 갖는, 측정 가능한 특성이다
 - Performance, Availability, Modifiability 등
- Scenario 기법으로 명세할 수 있다
 - Stimulus Source → Stimulus → Artifact(Environment) → Response → Response Measure



Quality Attribute Scenario 의 6개 요소

Constraints (제약사항)

■ 제약사항이란

- 설계에 주어진 한계나 제한
- 기술적 제약과 비기술적 제약이 있다
 - 기술적 예: open source 기술을 사용해야 한다
 - 비기술적 예: 12월 15일까지 납품해야 한다
- 고객뿐 아니라 개발 조직 내부에서 나오기도 한다

■ 제약사항을 대하는 자세

- 주어진 제약 안에서 가능한 최선의 시스템을 설계한다
- 제약을 완화해 달라고 설득할 수 있는 경우도 있지만, 대부분은 제약을 우회해서(design around) 설계할 수밖에 없다

Design Purpose (설계 목적)

■ 설계 목적에 따라 무엇을, 어떻게 설계할지가 달라진다

설계 상황	설계 목적과 특징
프로젝트 제안서의 일부로 설계	아주 상세할 필요가 없는 Architecture
탐색용 prototype 을 만드는 과정의 일부로 설계	출시하거나 재사용할 시스템을 만드는 것이 목적이 아니라, 도메인이나 새 기술을 탐색하는 것이 목적
개발 과정 중에 설계	요구사항을 만족시키고, 시스템 구축과 작업 배분을 이끌며, 최종 출시를 준비하기에 충분한 만큼 설계한다

Architectural Concerns (Architecture 관심사)

■ 정의

- Architecture 설계 과정에서 고려해야 하지만, 전통적인 요구사항 형태로는 표현되지 않는 추가적인 측면

■ 예시

- 전체적인 논리적·물리적 구조 만들기
- 입력 검증(Input validation)
- 예외 처리와 로깅(Exception management and logging)
- 통신(Communications)
- 배포와 업데이트(Deployment and updating)
- 데이터 이관과 백업(Data migration and backup)
- 코드베이스의 구성(Organization of the codebase)

Design Concepts (설계 개념)

■ 기존 해법을 재사용한다

- 설계 iteration 에서 다루는 하위 문제 대부분은 이미 존재하는 해법, 즉 Design Concept 으로 풀 수 있다
- 우리가 전문가가 아닌 문제라면, 검증된 해법을 쓰는 것이 더 좋고 더 빠르다
- 설계의 창의성은 Design Concept 을 찾아내고, 맞게 고치고, 조합하는 데서 나온다

■ Design Concept 의 종류 (추상적 → 구체적)

- Reference Architecture
- Deployment Pattern
- Architectural / Design Pattern
- Tactic
- Externally Developed Component (예: Framework)

핵심 검증된 Design Concept 을 찾아 조합하는 것이 Architecture 설계의 핵심 기술이다

Reference Architecture (참조 Architecture)

■ 정의

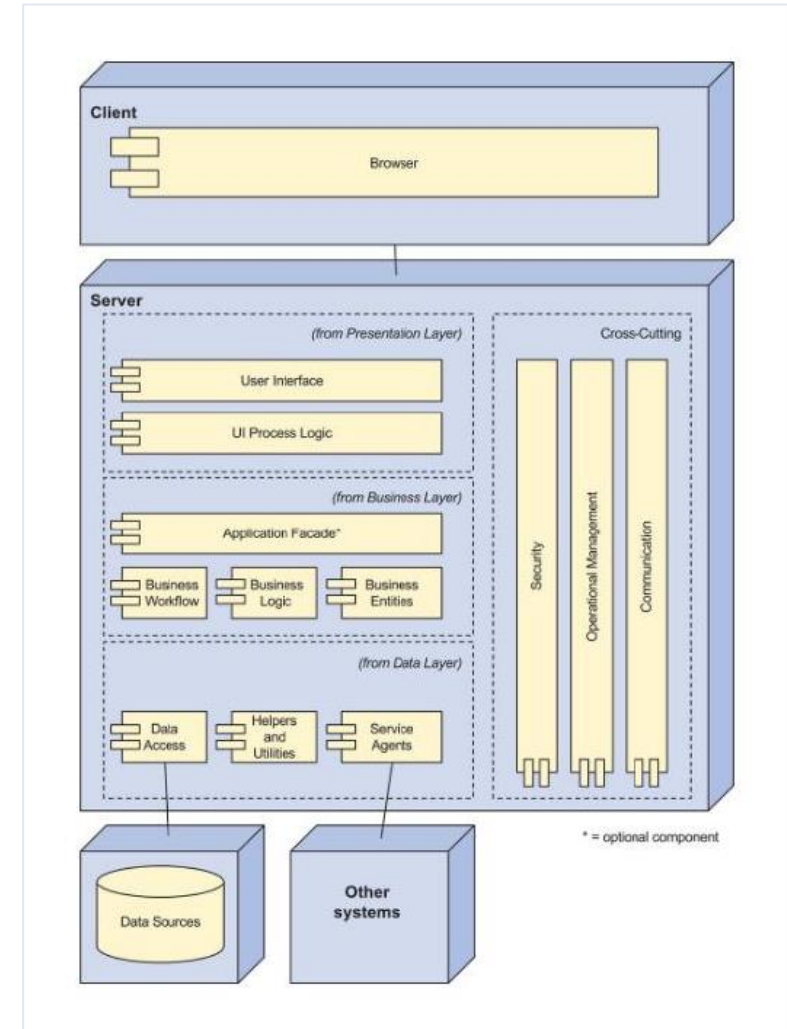
- 애플리케이션을 구조화하기 위한 청사진(blueprint)을 제공한다

■ 엔터프라이즈 애플리케이션 도메인의 예

- Mobile applications
- Rich client applications
- Rich internet applications
- Service applications
- Web applications

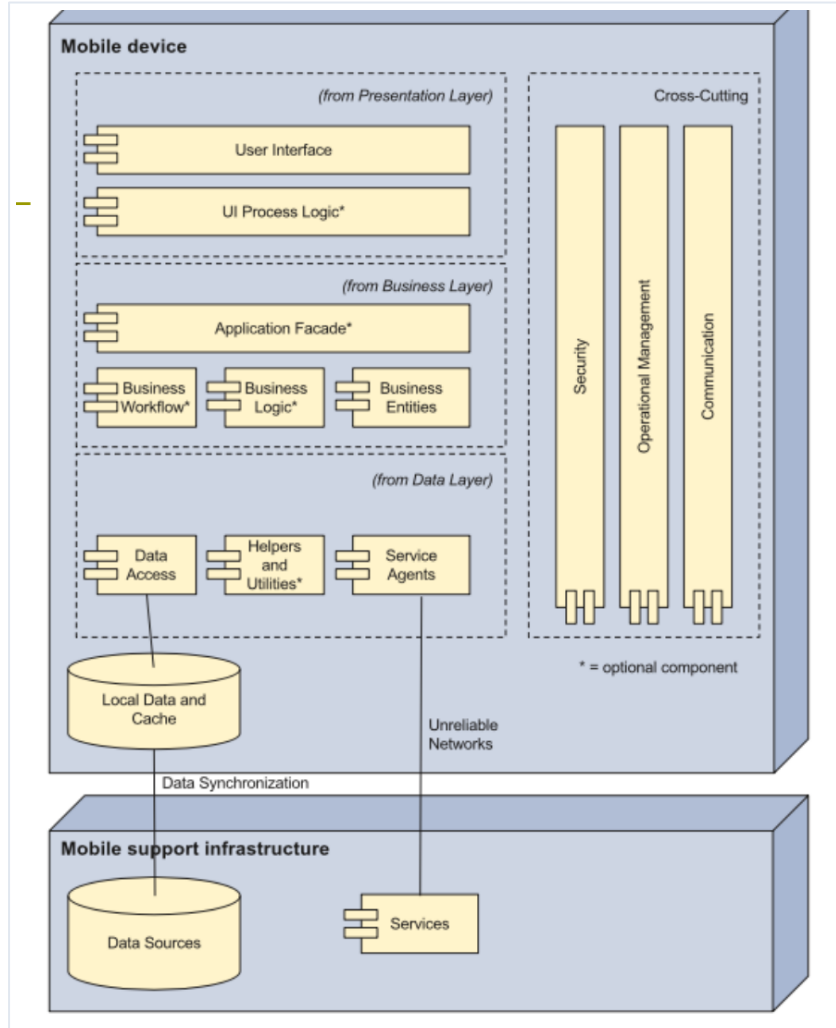
■ 오른쪽 그림

- Microsoft Application Architecture Guide 의 Web application 참조 Architecture
 - Client(Browser) 와 Server(Presentation / Business / Data Layer + Cross-Cutting)로 구성

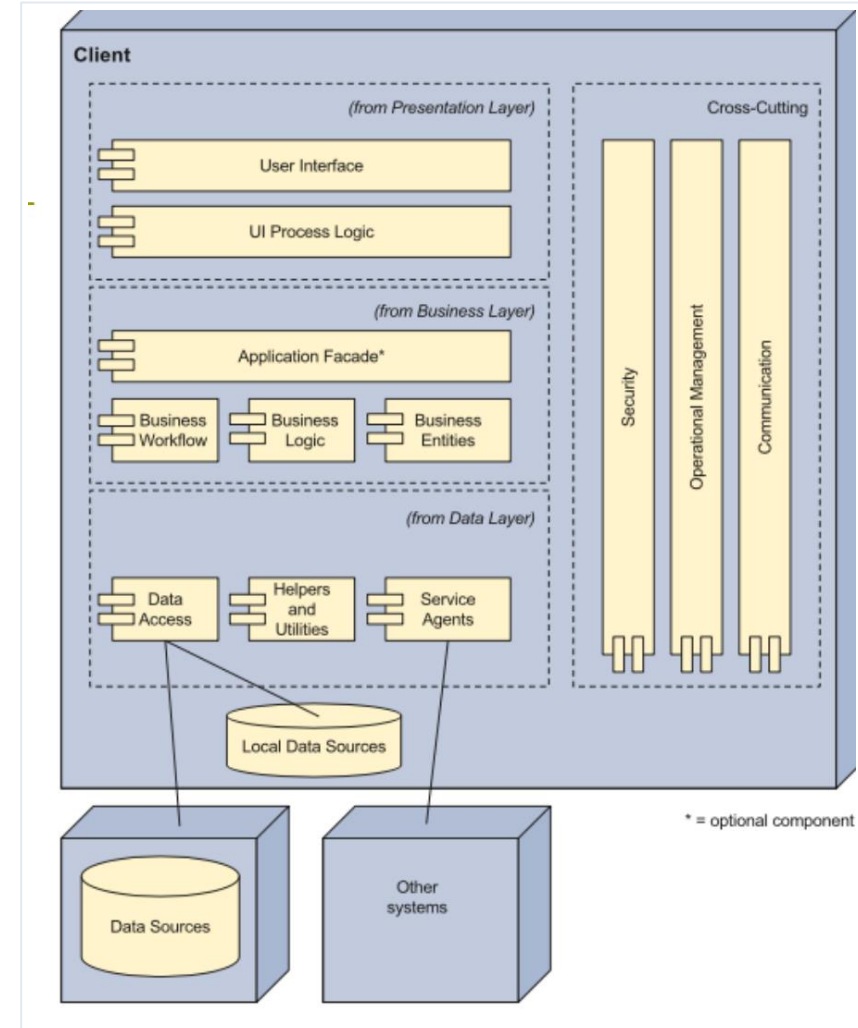


Web application 참조 Architecture (Microsoft Application Architecture Guide)

Reference Architecture 예: Mobile / Rich Client

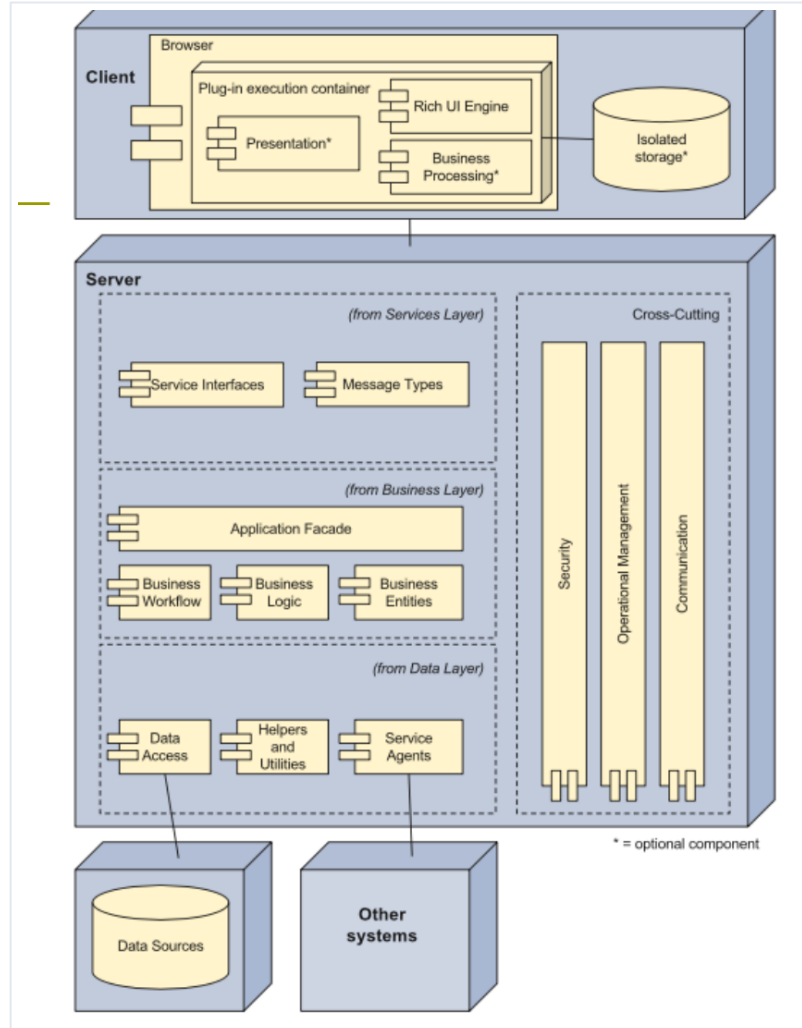


Mobile application: 기기 안에 Presentation / Business / Data Layer 와 Local Data Cache 를 두고 Mobile support infrastructure 와 동기화

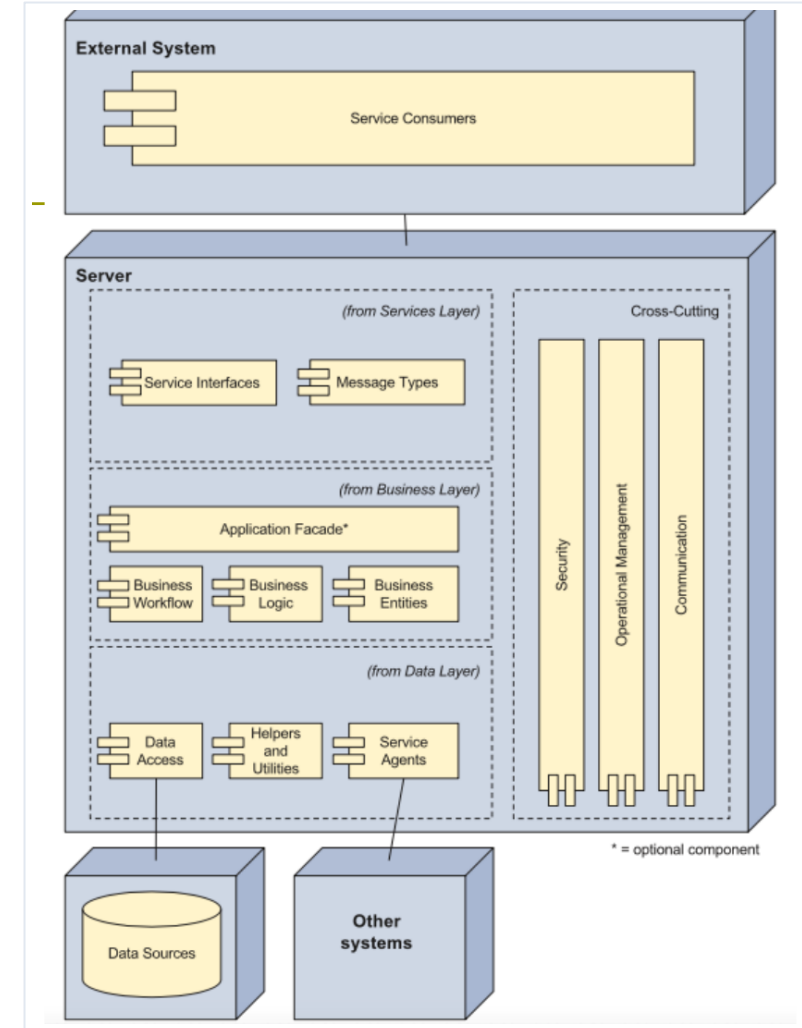


Rich client application: 사용자 PC 에 설치되며 Local Data Sources 와 외부 시스템을 사용

Reference Architecture 예: Rich Internet / Service



Rich internet application: Browser 의 Plug-in 실행 컨테이너 안에서 Rich UI Engine 이 동작, Server 는 Services Layer 제공



Service application: UI 없이 Services Layer 를 통해 외부 Service Consumer 에게 서비스를 노출

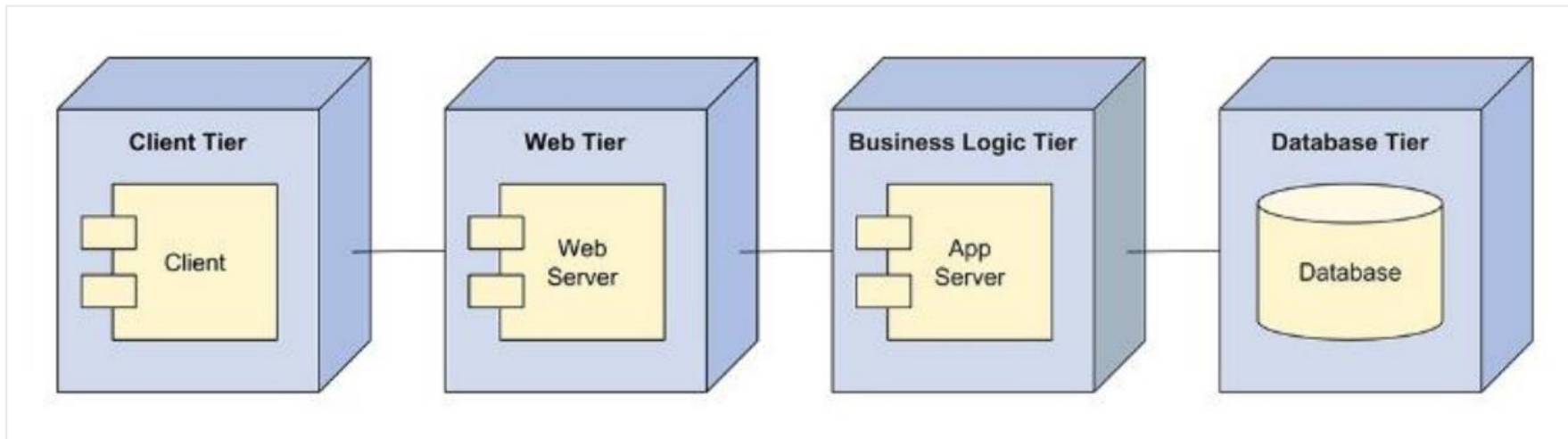
Deployment Pattern (배포 패턴)

■ 정의

- 시스템을 물리적 관점에서 어떻게 구조화할지 안내한다
- 소프트웨어 배포에 관한 좋은 결정은 Availability 같은 Quality Attribute 를 달성하는 데 필수적이다

■ 예

- 2-tier, 3-tier, 4-tier, n-tier deployment
- Load balanced cluster
- Failover cluster

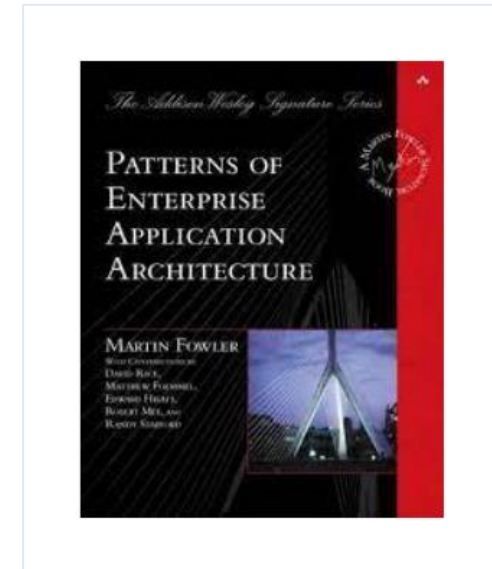
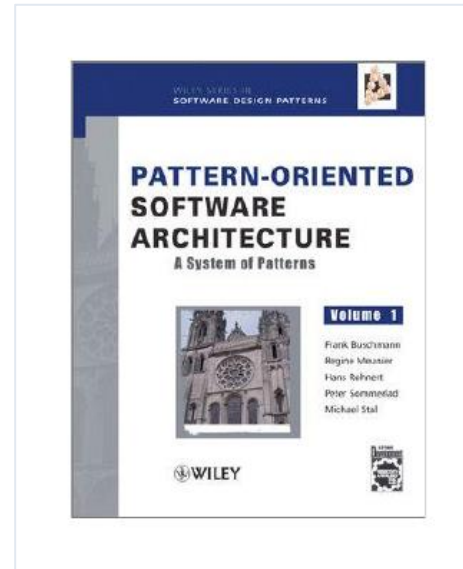
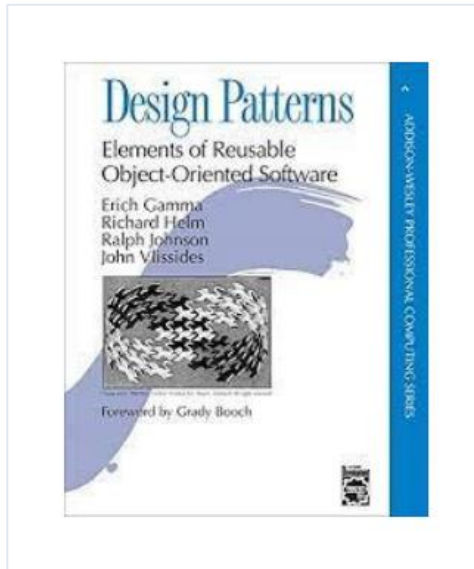


4-tier deployment pattern: Client Tier → Web Tier → Business Logic Tier → Database Tier (Microsoft Application Architecture Guide)

Architectural / Design Pattern

■ 정의와 특징

- Pattern 은 반복되는 설계 문제에 대한 검증된 (개념적) 해법이다
- 건축 분야에서 유래했다
- 수천 개의 패턴이 존재하며 여러 pattern catalog 에 문서화되어 있다
- "Design" pattern 과 "Architectural" pattern 사이에 명확한 경계를 긋기는 어렵다



Tactic (택틱)

■ 정의

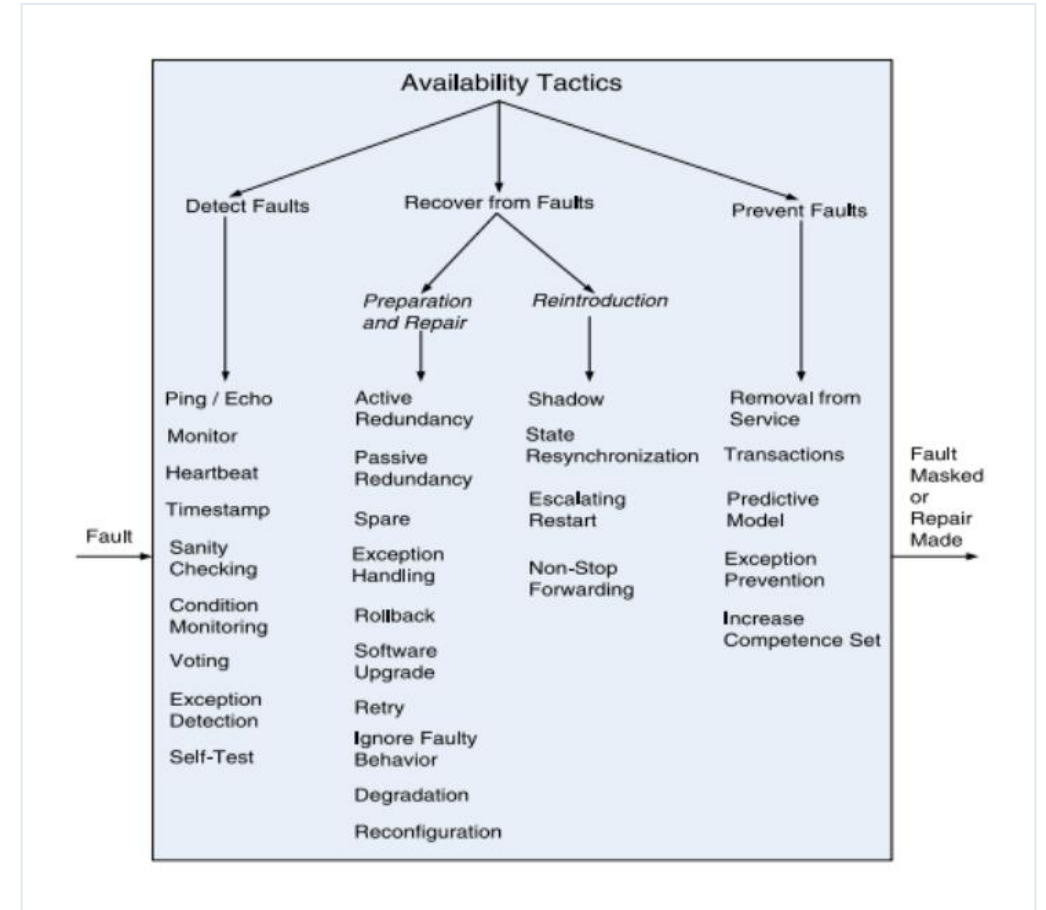
- Quality Attribute 의 response 를 제어하는 데 영향을 주는 설계 결정

■ Tactic 분류가 정리되어 있는 Quality Attribute

- Availability
- Interoperability
- Modifiability
- Performance
- Security
- Testability
- Usability

■ 오른쪽 그림

- Availability Tactics: Detect Faults / Recover from Faults / Prevent Faults 로 분류



Availability Tactics 분류 (3장에서 다룬 내용)

Externally Developed Component (외부 개발 컴포넌트)

■ 정의

- 재사용 가능한 코드 형태의 해법
 - 예: Middleware, Framework

■ Framework

- 여러 애플리케이션에 걸쳐 반복되는 관심사를 다루는, 범용 기능을 제공하는 재사용 소프트웨어 요소

Concern(관심사)	Framework (Java 예)	Usage(사용 방식)
로컬 사용자 인터페이스	Swing	Inheritance(상속)
Web UI	Java Server Faces (JSF)	XML, Annotations
Component 연결	Spring	XML, Annotations

Design Concept 찾기 (Identifying)

■ 대안을 여러 개 고려한다

- 적절한 설계 결정을 내리려면, 처음 떠오른 것 하나만이 아니라 iteration 목표를 만족하는 여러 대안을 검토해야 한다

방법	설명
기존 best practice 활용	인쇄물이나 온라인으로 제공되는 catalog 를 활용한다
자신의 지식과 경험 활용	과거에 설계한 시스템과 비슷하다면, 이전에 사용했던 Design Concept 에서 출발할 수 있다
다른 사람의 지식과 경험 활용	동료들과의 brainstorming 을 통해 Design Concept 을 찾아낸다

Design Concept 선택하기 (Selecting)

■ 장단점 표로 비교한다

- 대안 Design Concept 목록을 찾았으면, 당면한 설계 문제를 풀 하나를 골라야 한다
- 각 대안의 장점(Pros)과 단점(Cons), 비용(Cost)을 표로 정리해 비교한다

대안	Pros(장점)	Cons(단점)	Cost
Web application	표준 web browser 로 다양한 플랫폼에서 접근 가능 / 빠른 페이지 로딩 / 간단한 배포	"rich" 한 상호작용을 지원하지 못함	Low
Rich Internet application	"rich" 한 사용자 상호작용 지원 / 간단한 배포와 업데이트	페이지 로딩 시간이 길다 / client browser 에 runtime 환경 설치 필요	Medium
Mobile application	"rich" 한 사용자 상호작용 지원	이식성이 낮다 / 화면 크기 제약	High

Attribute-Driven Design (ADD) 란?

■ 정의

- ADD 는 Quality 요구사항과 기능 요구사항, 그리고 Constraint 를 모두 만족하는 Architecture 를 설계하기 위한 방법(method)이다
 - Version 2.0: 2006년 11월 발표
 - Version 3.0: 2016년 5월 발표

■ 특징

- 반복적(iterative) 설계 접근법을 권장한다
- Architecture 설계를 위한 상세한 단계(step)를 제공한다

핵심 ADD = Quality Attribute 를 중심 Driver 로 삼아, 정해진 단계를 반복하며 Architecture 를 만드는 방법

ADD 3.0 의 단계와 산출물

■ 입력(Driver)

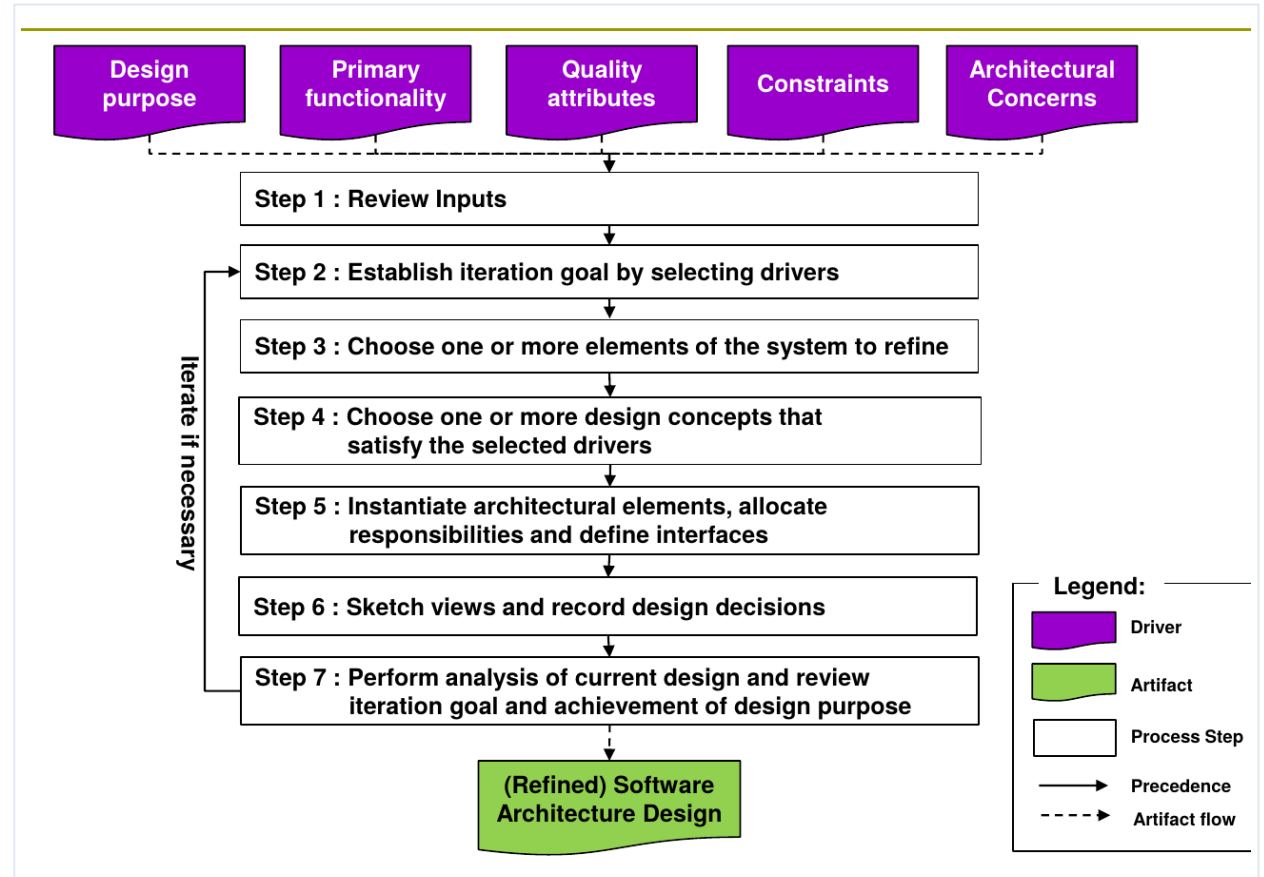
- Design purpose, Primary functionality, Quality attributes, Constraints, Architectural Concerns

■ 7단계

- Step 1: 입력 검토
- Step 2: Driver 를 선택해 iteration 목표 수립
- Step 3: 정제할 시스템 요소 선택
- Step 4: Driver 를 만족하는 Design Concept 선택
- Step 5: 요소 인스턴스화, 책임 할당, 인터페이스 정의
- Step 6: View 스케치와 설계 결정 기록
- Step 7: 현재 설계 분석, iteration 목표와 설계 목적 달성 여부 검토

■ 출력

- (정제된) Software Architecture Design — Step 2~7 을 필요한 만큼 반복



ADD 3.0 의 단계와 산출물 (보라: Driver, 초록: Artifact, 흰색: Process Step)

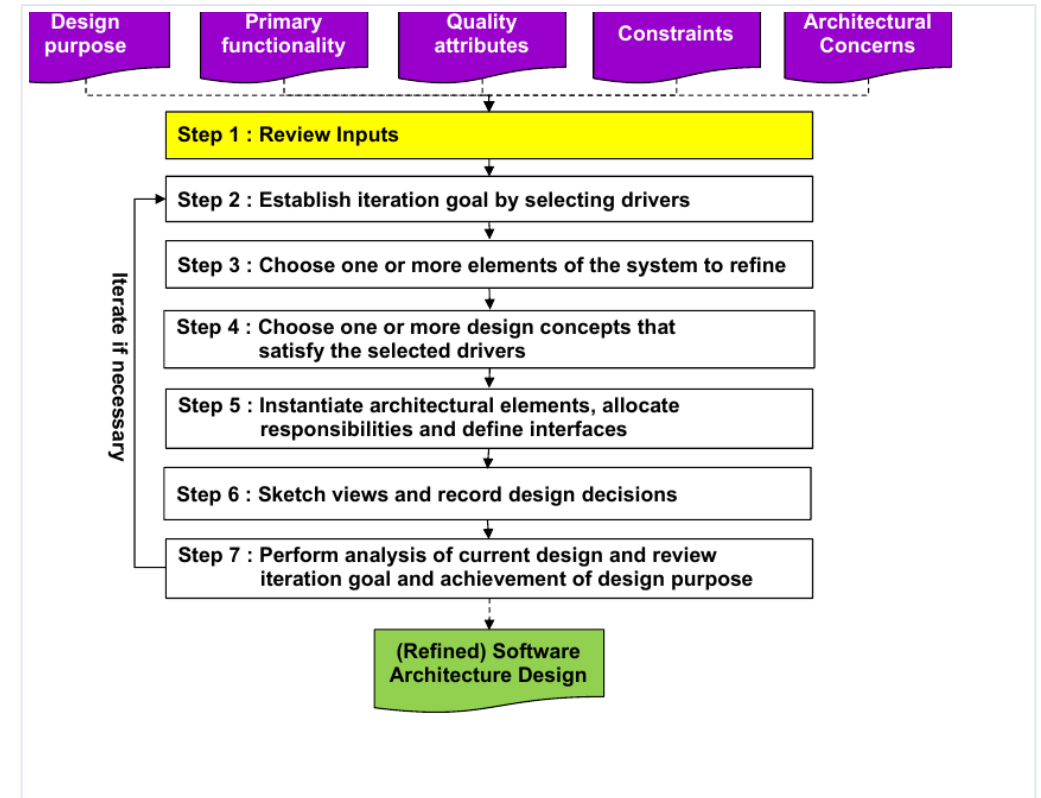
Step 1: Review Inputs (입력 검토)

■ 설계를 시작하기 전에 확인할 것

- 설계 round 를 시작하기 전에, 설계 프로세스의 입력이 모두 준비되어 있고 올바른지 확인한다
- 이 시점에는 Primary functionality 와 Quality Attribute Scenario 의 우선순위가 이미 정해져 있다고 가정한다

■ 설계 시작 조건

- 주요 use case 와 가장 중요한 Quality Attribute Scenario 가 정해져 있으면 설계를 시작할 수 있다



Step 1 — 5가지 Driver 를 입력으로 검토

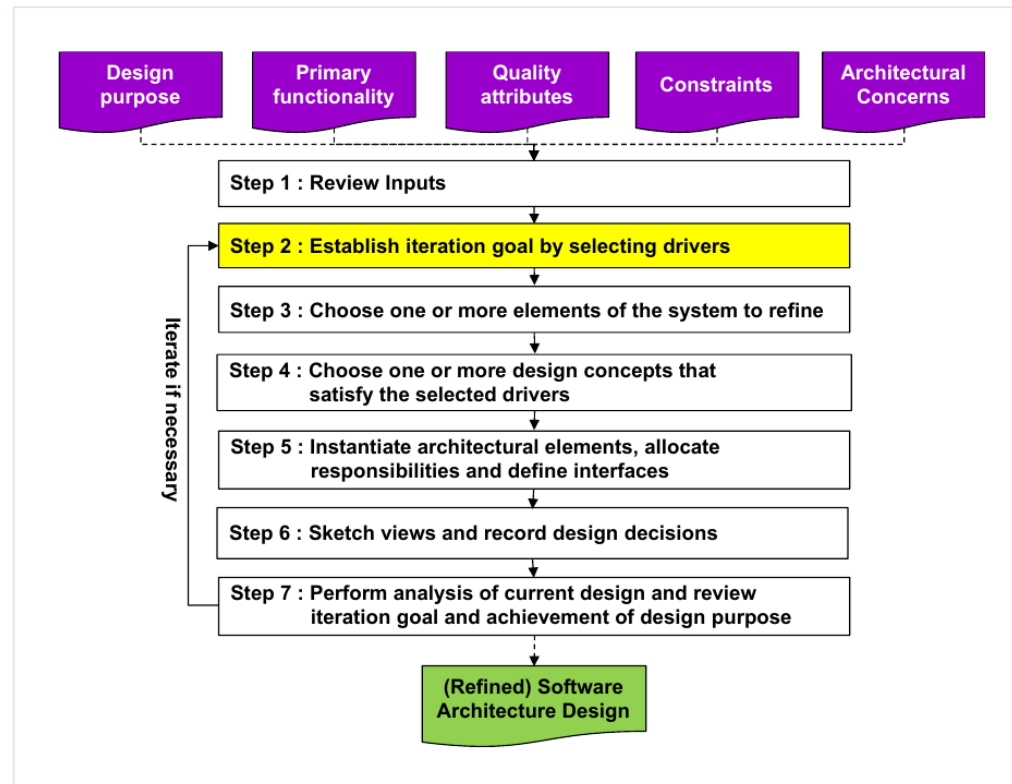
Step 2: Establish Iteration Goal (반복 목표 수립)

■ iteration 마다 하나의 목표

- 설계 round 는 보통 여러 번의 설계 iteration 으로 수행되며, 각 iteration 은 특정 목표 하나를 달성하는 데 집중한다
 - 예: 전체 구조 잡기, 주요 기능 지원 구조 찾기, 특정 QA Scenario 다루기

■ 목표의 순서

- 설계하는 시스템의 유형에 따라, 다루어야 할 iteration 목표에 "가장 좋은" 순서가 있을 수 있다

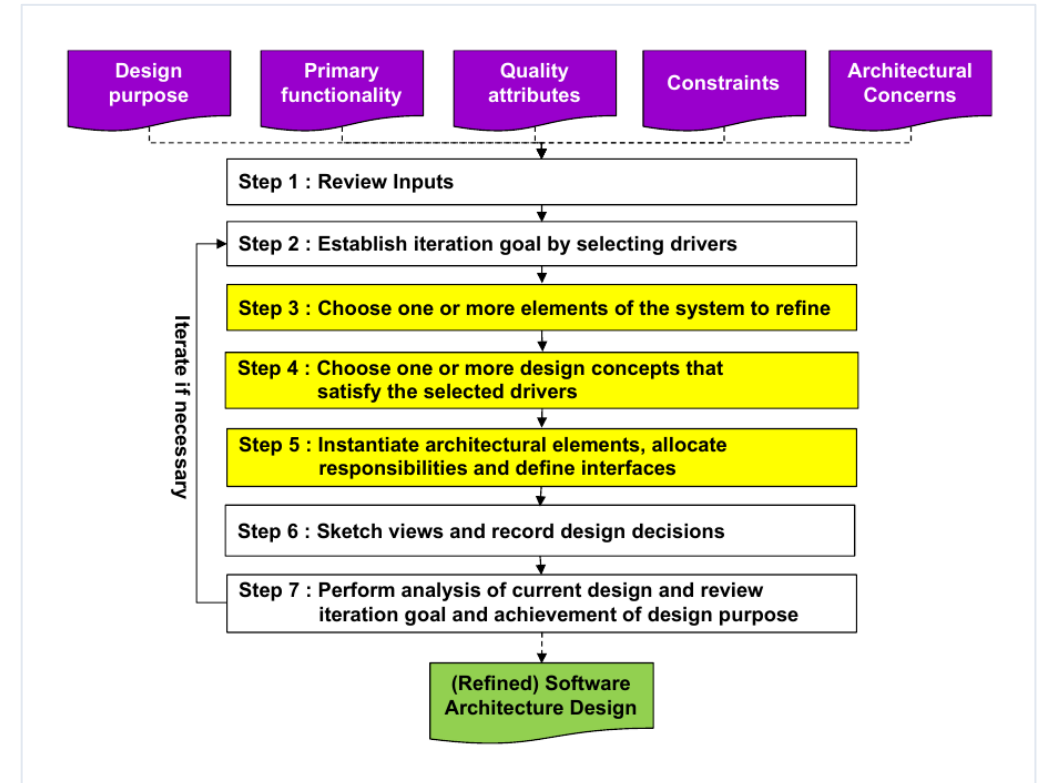


Step 2 — Driver 를 골라 iteration 목표를 세운다

Step 3/4/5: Make Design Decisions (설계 결정)

■ iteration 목표를 위해 내리는 결정의 종류

- Step 3: 분해(decompose)할 부분을 선택한다
- Step 4: 분해를 뒷받침하는 기존 해법(Design Concept)을 찾아 선택한다
- Step 5-1: 기존 해법으로부터 요소(element)를 인스턴스화한다
- Step 5-2: 각 요소에 책임(responsibility)을 할당한다
- Step 5-3: 협력하는 요소들 사이의 인터페이스를 정한다



Step 3~5 — 요소 선택 → Design Concept 선택 → 인스턴스화.책임.인터페이스

핵심 설계 결정의 핵심은 "무엇을 나누고, 어떤 검증된 해법으로, 어떻게 채울 것인가" 이다

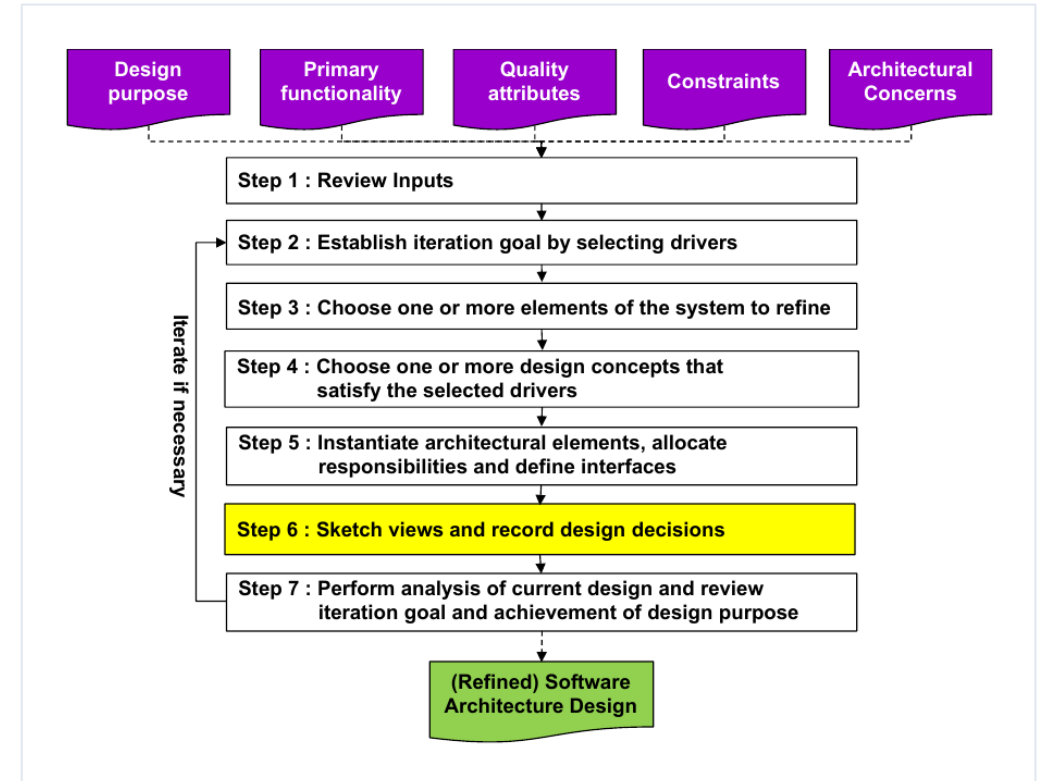
Step 6: Sketch Views and Record Design Decisions

■ View 스케치

- 이 단계는 Step 5 와 병행해서 수행할 수 있다
- 만들어 낸 구조를 표현한 View 를 반드시 기록으로 남긴다
 - 이 View 는 거의 확실히 불완전하므로, 이후 iteration 에서 다시 보고 다듬어야 한다

■ 설계 결정 기록

- 중요한 결정과 그 결정에 이르게 된 이유(Rationale)를 기록한다
 - 나중에 분석하고, 결정을 이해하기 쉽게 하기 위해서다



Step 6 — View 를 스케치하고 결정과 근거를 기록

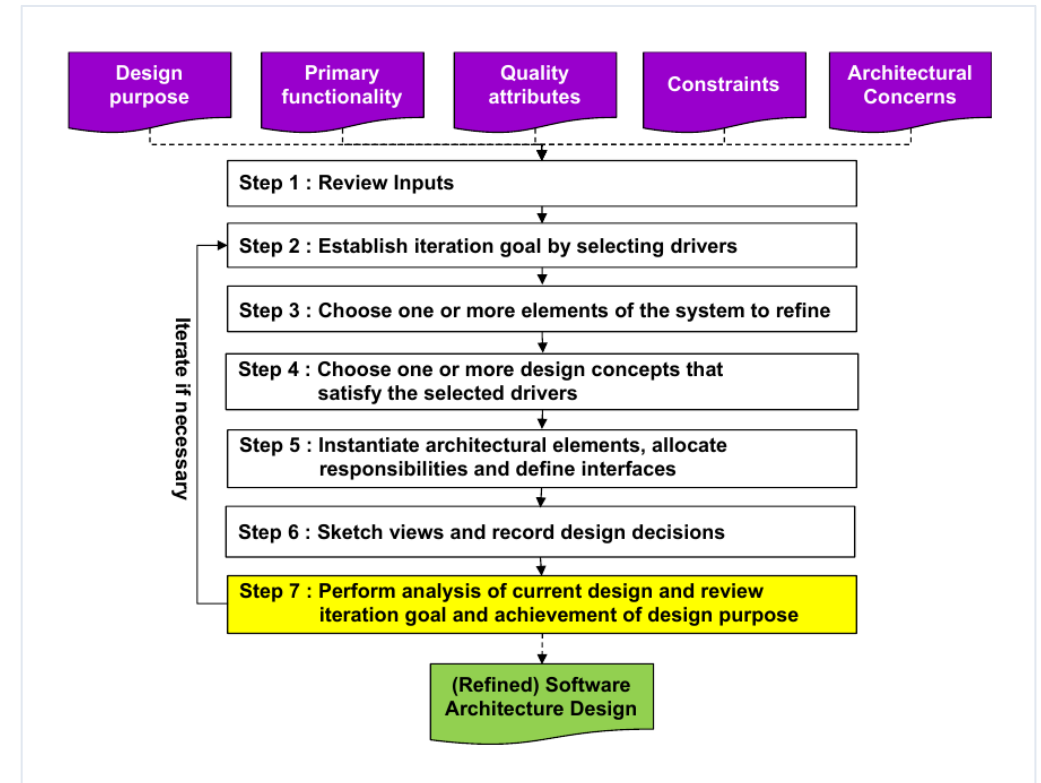
Step 7: 현재 설계 분석과 목표 달성 검토

■ 현재 설계 분석

- 기록해 둔 View 스케치와 설계 결정을 검토하면서 스스로 분석할 수 있다
- 그러나 다른 사람이 설계 검토를 도와주도록 하는 것이 더 좋은 방법이다

■ 목표 달성 검토

- iteration 에서 수행한 설계를 분석했으면, 처음에 세운 Design Purpose(설계 목적) 관점에서 Architecture 의 현재 상태를 검토한다
- 필요하면 Step 2 로 돌아가 다음 iteration 을 수행한다



Step 7 — 분석·검토 후 필요하면 다시 반복

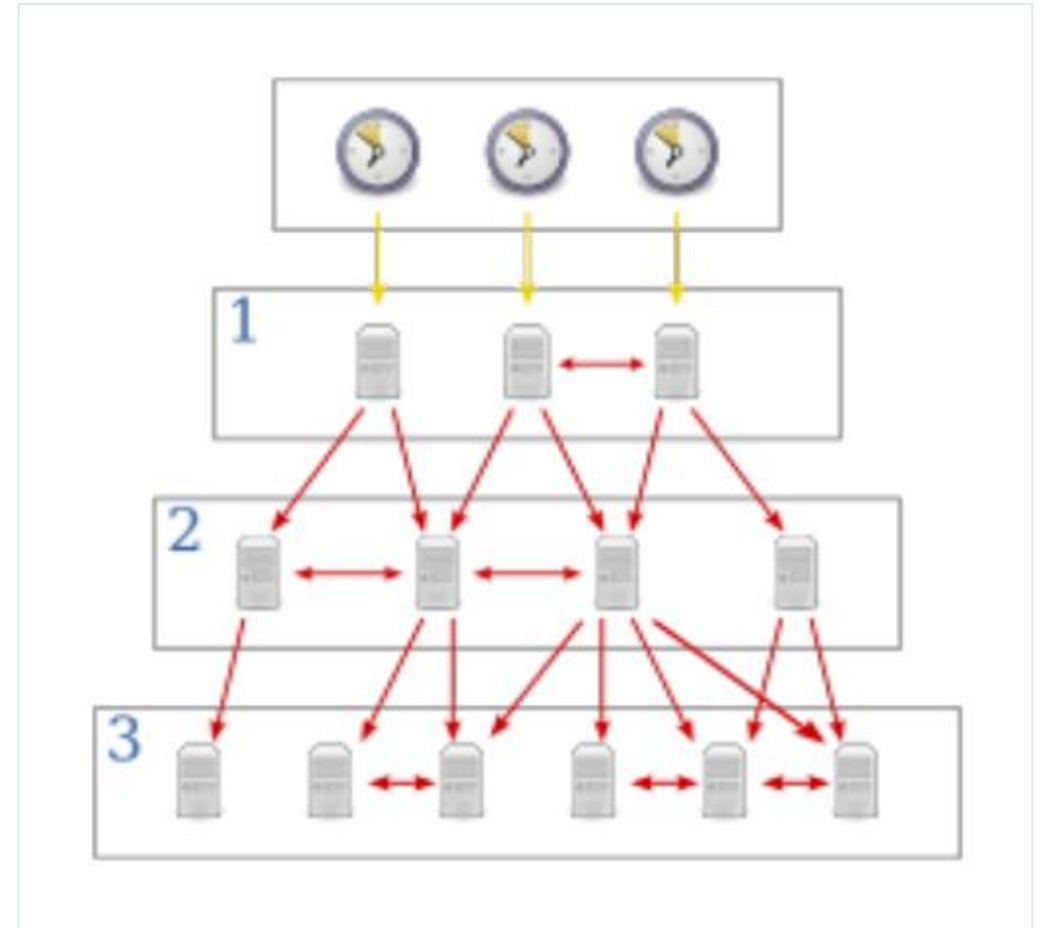
Case Study: FCAPS System

■ 대상 시스템

- Time server 네트워크를 관리하는 관리 시스템 (management system)
 - Time server: 기준 시계(reference clock)에서 실제 시각을 읽어 네트워크를 통해 client 들에게 배포하는 서버 컴퓨터

■ FCAPS 의 뜻

- Fault management (장애 관리)
- Configuration management (구성 관리)
- Accounting (과금/계정)
- Performance management (성능 관리)
- Security management (보안 관리)



기준 시계로부터 계층적으로 시각을 배포하는 Time server 네트워크

요구사항(Requirements) 개요

■ 수집된 요구사항

- 요구사항 도출(elicitation) 활동은 이미 수행되었고, 가장 관련 있는 요구사항을 다음 네 가지로 정리했다
- Use Case Model
- Quality Attribute Scenarios
- Constraints
- Architectural Concerns

■ 출처

- 이후 그림과 표는 다음 책에서 가져왔다
 - H. Cervantes, R. Kazman, Designing Software Architectures: A Practical Approach, Addison Wesley, 2016

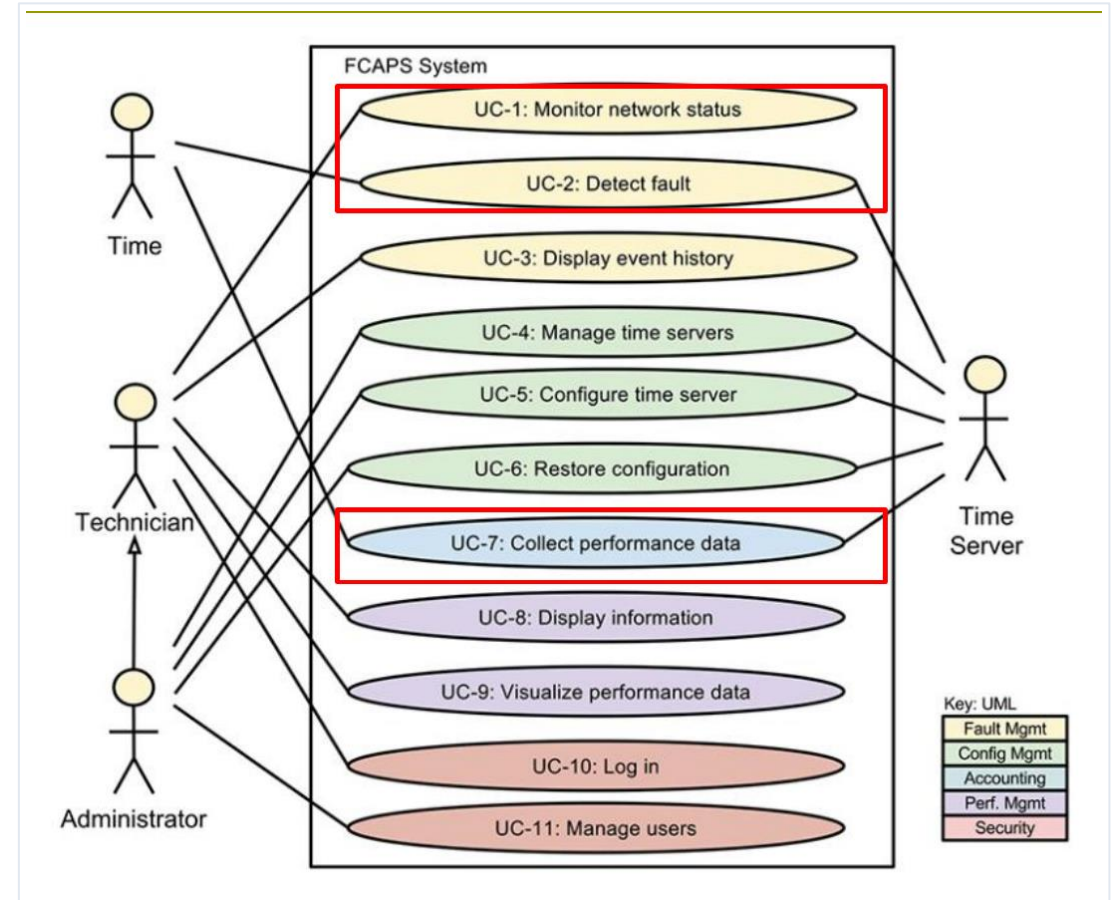
Use Case Model

■ Actor

- Time Server, Technician, Administrator

■ Use Case (11개)

- UC-1 네트워크 상태 모니터링, UC-2 장애 탐지 (Fault Mgmt)
- UC-3 이벤트 이력 표시, UC-4~6 time server 관리·구성·구성 복원 (Config Mgmt)
- UC-7 성능 데이터 수집, UC-8~9 정보 표시·시각화 (Perf Mgmt)
- UC-10 로그인, UC-11 사용자 관리 (Security)
- 빨간 테두리(UC-1, UC-2, UC-7)가 Primary use case



FCAPS System 의 Use Case Model

Quality Attribute Scenarios

- QA-1 ~ QA-4 (빨간 테두리)가 나중에 Driver 로 선택된다

ID	QA	Scenario	관련 UC
QA-1	Performance	여러 time server 가 peak load 에서 trap 을 보내면, 관리 시스템은 trap 의 100% 를 성공적으로 처리하고 저장한다	UC-2
QA-2	Modifiability	업데이트로 새로운 time server 관리 protocol 이 도입되면, 핵심 component 를 바꾸지 않고 protocol 을 추가할 수 있다	UC-5
QA-3	Availability	정상 운영 중 관리 시스템에 장애가 발생하면, 30초 이내에 운영을 재개한다	All
QA-4	Performance	peak load 에서 time server 의 성능 데이터를 수집할 때, 모든 사용자 요청을 처리하면서 5분 안에 모든 성능 데이터를 수집한다(CON-5 로 인한 데이터 손실 방지)	UC-7
QA-5	Performance, Usability	정상 운영 중 사용자가 특정 time server 의 이벤트 이력을 표시하면, 최근 24시간의 이벤트 목록이 1초 안에 표시된다	UC-3
QA-6	Security	정상 운영 중 사용자가 시스템을 변경하면, 누가 언제 변경했는지 100% 알 수 있다	All

Constraints 와 Architectural Concerns

ID	내용
CON-1	최소 50명의 동시 사용자를 지원해야 한다
CON-2	여러 플랫폼(Windows, OSX, Linux)에서 web browser(Chrome V3.0+, Firefox V4+, IE8+)로 접근해야 한다
CON-3	기존 관계형 database server 를 사용해야 하며, 이 서버는 database 호스팅 외 용도로 쓸 수 없다
CON-4	사용자 workstation 과의 네트워크 연결은 대역폭이 낮을 수 있지만 대체로 신뢰할 만하다
CON-5	성능 데이터는 5분 이하 간격으로 수집해야 한다(간격이 길면 time server 가 데이터를 버린다)
CON-6	최근 30일의 이벤트를 저장해야 한다
CRN-1	전체적인 초기 시스템 구조를 수립한다
CRN-2	팀의 Java 기술 지식(Spring, JSF, Swing, Hibernate, Java Web Start, JMS, Java 언어)을 활용한다
CRN-3	개발 팀원들에게 작업을 배분한다

Step 1: Review Inputs — Driver 정리

- 설계를 시작하기 전에 5가지 Driver 를 표로 정리하고, 어떤 것을 이번 설계에서 다룰지 결정한다

Category	Details
Design purpose	시스템 구축을 뒷받침할 수 있을 만큼 충분히 상세한 설계를 만드는 것
Primary functional requirements	Use case 중 UC-1, UC-2, UC-7 을 primary 로 결정
Quality attribute scenarios	고객 중요도 / 구현 난이도로 우선순위를 매김 — QA-1 High/High, QA-2 High/Medium, QA-3 High/High, QA-4 High/High, QA-5 Medium/Medium, QA-6 Medium/Low → QA-1, QA-2, QA-3, QA-4 를 Driver 로 선택
Constraints	앞에서 제시한 제약사항 전부를 Driver 에 포함
Architectural concerns	앞에서 제시한 관심사 전부를 Driver 에 포함

Iteration 계획

■ Iteration 1

- 전체 시스템 구조 수립
- (Establishing an Overall System Structure)
- Architectural Concern CRN-1 을 다룬다

■ Iteration 2

- 주요 기능을 지원하는 구조 식별
- (Identifying Structures to Support Primary Functionality)
- UC-1, UC-2, UC-7 을 다룬다

■ Iteration 3

- Quality Attribute Scenario Driver 다루기
- (Addressing QA-3: Availability)
- Application server, Database server 를 정제한다

핵심 각 iteration 은 ADD 의 Step 2~7 을 한 바퀴 수행한다

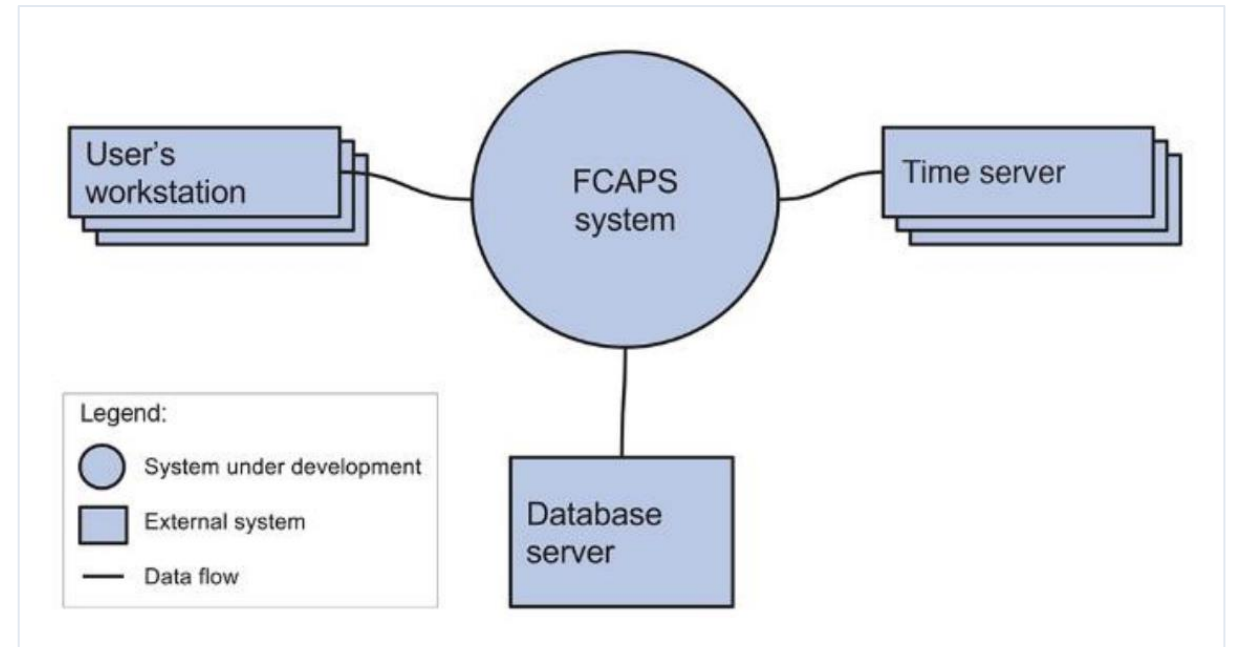
Iteration 1 — Step 2·3: 목표 수립과 정제 요소 선택

■ Step 2: Iteration 목표

- greenfield(신규) 시스템 설계의 첫 iteration 이므로, 목표는 Architectural Concern CRN-1 달성
 - 전체적인 시스템 구조를 수립하는 것

■ Step 3: 정제할 요소

- 신규 개발이므로 정제할 요소는 FCAPS 시스템 전체
- 외부 시스템: User's workstation, Time server, Database server



Context: FCAPS system(개발 대상)과 외부 시스템

Iteration 1 — Step 4: Design Concept 선택 (1)

- 첫 iteration 의 목표는 시스템 전체를 구조화하는 것이므로, 그에 맞는 Design Concept 을 선택한다

설계 결정	근거(Rationale)
Client 부분을 Rich Client Application 참조 Architecture 로 논리적 구조화	사용자 PC 에 설치되는 애플리케이션 개발을 지원하며, UC-1 에 필요한 rich UI 를 지원한다. web browser 안에서 실행되지는 않지만(CON-2) Java Web Start 로 browser 에서 설치할 수 있다
버린 대안: Rich Internet Application (RIA)	browser 안에서 실행되는 rich UI 를 지원하지만, RIA 실행용 plugin 이 JVM 보다 널리 보급되어 있지 않아 제외
버린 대안: Web application	rich 한 사용자 경험을 제공하기 어려워 제외

Iteration 1 — Step 4: Design Concept 선택 (2)

설계 결정	근거(Rationale)
Server 부분을 Service Application 참조 Architecture 로 논리적 구조화	Service application 은 UI 없이 다른 애플리케이션이 사용하는 서비스를 노출한다. Architect 가 이 참조 Architecture 에 익숙하고 요구사항에 충분하다고 판단해 다른 대안은 고려하지 않았다
3-tier deployment pattern 으로 물리적 구조화	web browser 로 접근해야 하고(CON-2) 기존 database server 를 써야 하므로(CON-3) 3-tier 배포가 적절하다. 버린 대안: $n \neq 3$ 인 다른 n-tier pattern
Client 애플리케이션의 UI 를 Swing Java framework 로 구축	Java Rich Client 의 표준 framework 로 이식성을 보장하고(CON-2) 개발자들이 이미 익숙하다(CRN-2). 버린 대안: Eclipse SWT — 개발자들이 덜 익숙함

Iteration 1 — Step 5: 요소 인스턴스화와 책임 할당

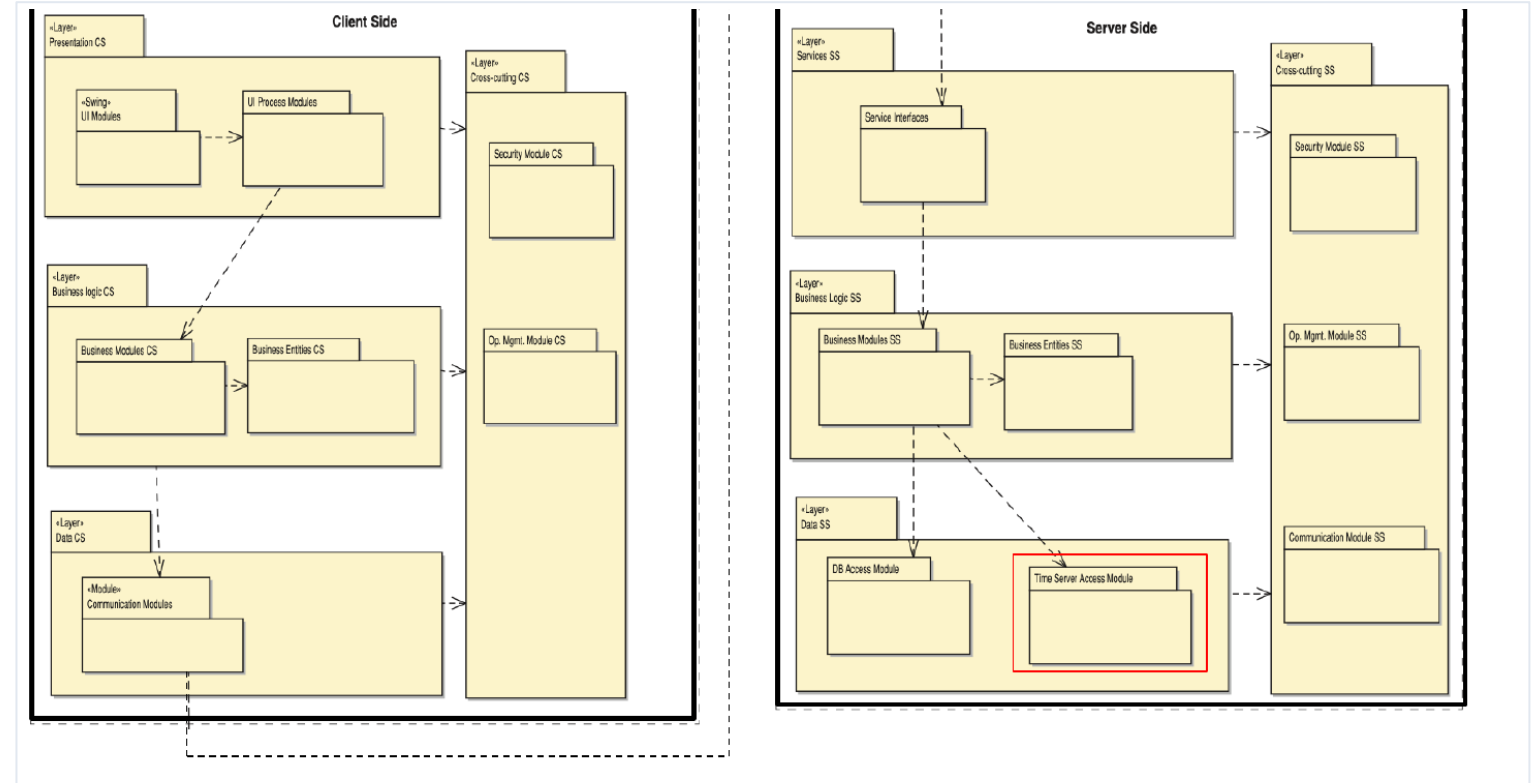
- 검토하고 결정한 인스턴스화 설계 결정을 표로 정리한다
- 첫 iteration 에서는 기능과 인터페이스를 정확히 정의하기에는 보통 너무 이르다

설계 결정과 위치	근거(Rationale)
Rich client application 에서 local data source 제거	네트워크 연결이 대체로 신뢰할 만하므로(CON-4) 데이터를 로컬에 저장할 필요가 없다. 서버와의 통신은 data layer 가 담당하고, client 내부 통신은 local method call 로 처리하므로 별도 지원이 필요 없다
Service Application 참조 Architecture 의 data layer 에 time server 접근 전용 module 생성	참조 Architecture 의 service agents component 를 time server 접근을 추상화하도록 조정한다. QA-2 달성을 돕고, UC-2 와 UC-7 달성에 핵심 역할을 한다

Iteration 1 — Step 6: Module View 스케치

■ Module view

- Client Side: Presentation / Business Logic / Data Layer 와 Cross-cutting (Security, Op. Mgmt) module
- Server Side: Services / Business Logic / Data Layer 와 Cross-cutting module
- Server 의 Data Layer 에 Time Server Access Module(빨간 테두리)을 추가

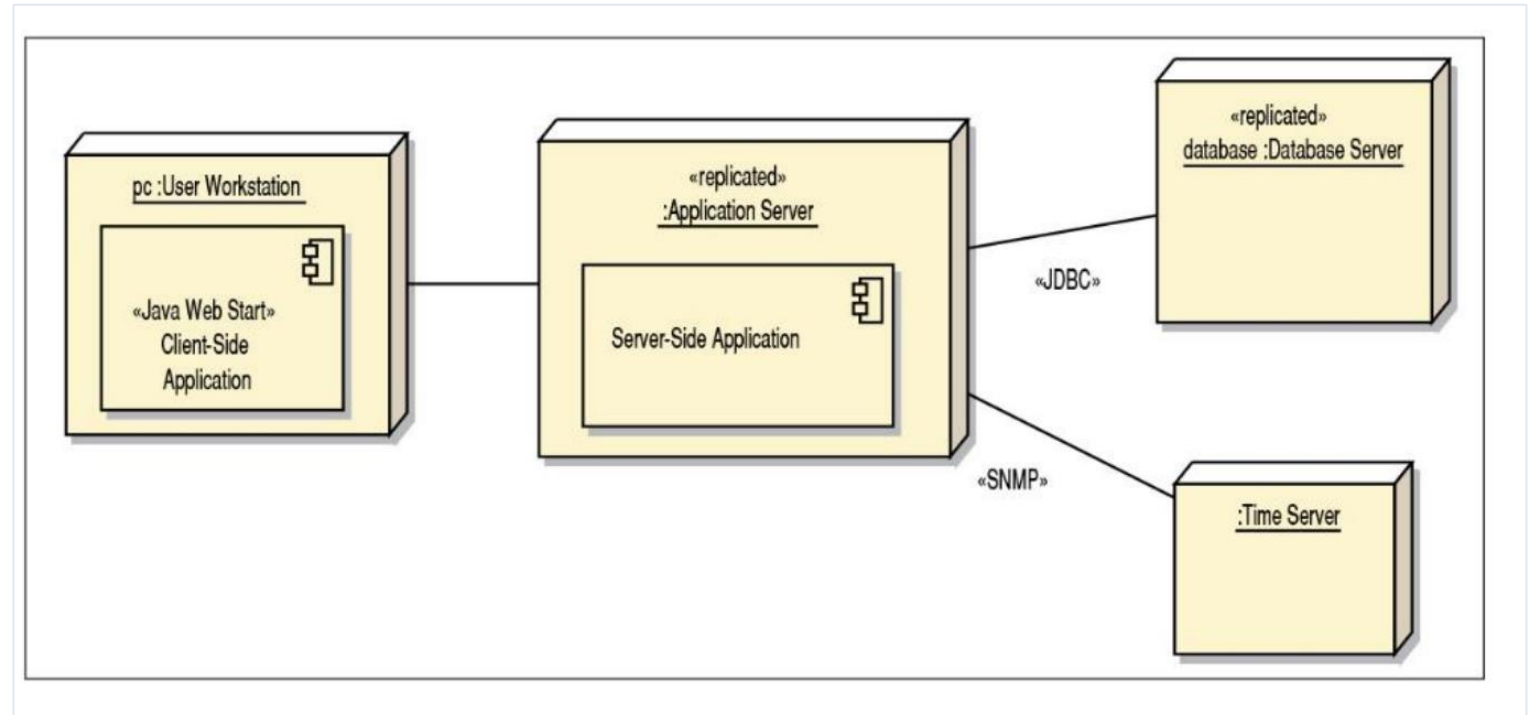


Iteration 1 의 Module View 스케치 (Client Side / Server Side)

Iteration 1 — Step 6: Deployment(Allocation) View 스케치

■ Deployment diagram

- 앞의 module 에 대응하는 component 가 어디에 배포되는지 보여주는 Allocation View
- User Workstation(Java Web Start Client) — Application Server(replicated) — Database Server(replicated, JDBC) — Time Server(SNMP)



Iteration 1 의 Deployment diagram (3-tier)

Iteration 1 — Step 7: 분석과 검토 (1) UC·QA

Driver	상태	이번 iteration 의 설계 결정
UC-1	부분 해결	선택한 참조 Architecture 가 이 기능을 지원할 module 을 정해 준다
UC-2	부분 해결	선택한 참조 Architecture 가 module 을 정해 준다 (...)
QA-1	미해결	관련 결정 없음 — 이 scenario 와 연관된 use case 에 참여하는 요소를 먼저 식별해야 한다
QA-2	부분 해결	서버 애플리케이션의 data layer 에 time server 와의 통신을 캡슐화하는 time server access module 도입. 이 component 의 세부 사항과 인터페이스는 아직 정의되지 않음
QA-3	부분 해결	deployment pattern 에서 도출된 요소 중 복제(replicate)가 필요한 요소를 식별

Iteration 1 — Step 7: 분석과 검토 (2) CON·CRN

Driver	상태	이번 iteration 의 설계 결정
CON-1	부분 해결	3-tier 구조로 여러 client 가 application server 에 접속 가능. 동시 접근에 관한 결정은 아직 없음
CON-2	부분 해결	Java Web Start 로 web browser 를 통해 Rich Client 를 내려받아 접근. Java 로 작성되므로 Windows, OSX, Linux 에서 실행 가능
CON-3	부분 해결	3-tier deployment pattern 으로 물리적 구조화, application server 의 data layer 에 database 접근 component 를 둠
CON-4	부분 해결	Java Web Start 기술 사용 (...)
CRN-1	완전 해결	참조 Architecture 와 deployment pattern 선택
CRN-2	부분 해결	지금까지 고려한 기술은 개발자의 지식을 반영. 다른 기술(예: time server 와의 통신)은 아직 선택 필요
CRN-3	미해결	관련 결정 없음

Iteration 2 — Step 2·3: 목표 수립과 정제 요소 선택

■ Iteration 2 의 방향

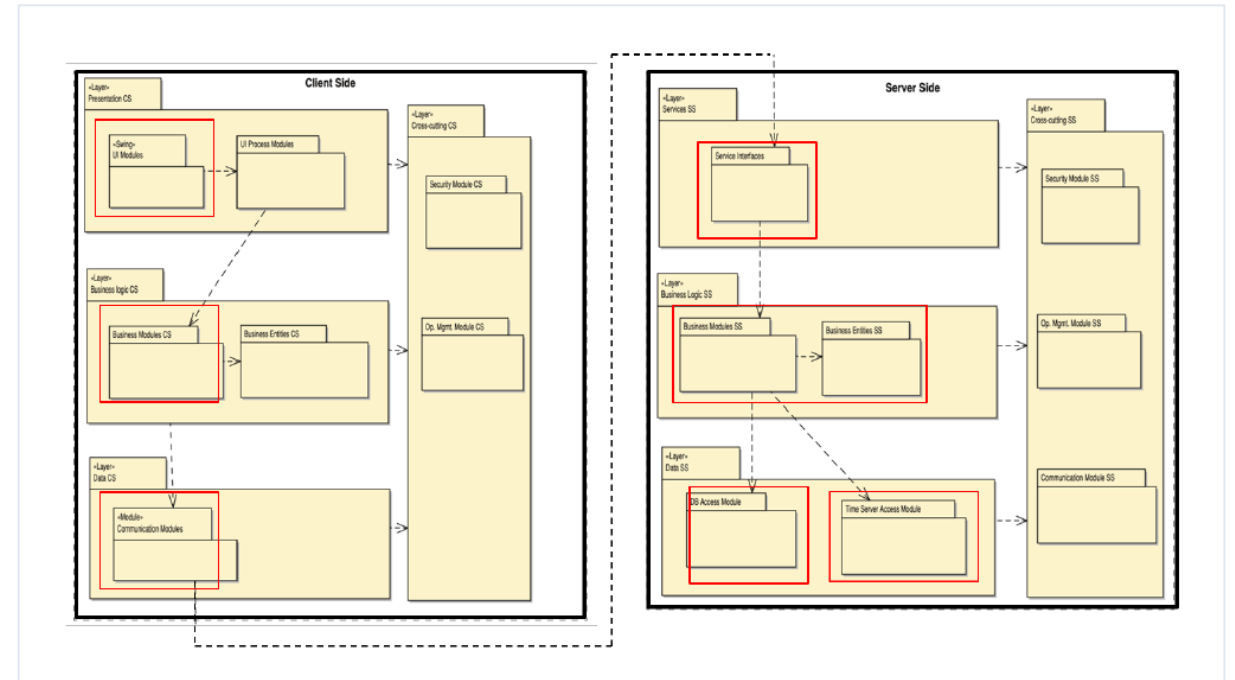
- Iteration 1 의 일반적이고 거친(coarse-grained) 기능 설명에서, 구현과 개발 팀 구성을 이끌 만큼 상세한 결정으로 나아간다

■ Step 2: Iteration 목표

- "주요 기능을 지원하는 구조 식별" 이라는 일반적인 Architectural Concern 을 다룬다
- 시스템의 primary use case 인 UC-1, UC-2, UC-7 을 고려한다

■ Step 3: 정제할 요소

- 이전 iteration 의 두 참조 Architecture 가 정의한 여러 layer 에 위치한 module 들(빨간 테두리)



정제 대상 module (Client Side / Server Side 의 각 layer)

Iteration 2 — Step 4: Design Concept 선택 (1)

설계 결정	근거(Rationale)
애플리케이션의 Domain Model 생성	기능 분해를 시작하기 전에, 도메인의 주요 entity 와 그 관계를 식별하는 초기 domain model 을 만들어야 한다. 좋은 대안은 없다
기능 요구사항에 대응되는 Domain Object 식별	애플리케이션의 각 기능 요소는 자기 완결적인 building block, 즉 domain object 로 캡슐화해야 한다. Domain object 를 고려하지 않고 layer 를 바로 module 로 분해할 수도 있지만, 요구사항을 놓칠 위험이 커진다
Domain Object 를 특화된 Module 로 분해	Domain object 는 완전한 기능 집합을 나타내지만, 이 기능은 각 layer 안의 더 세분화된 요소가 지원한다. Module 의 특화는 그 module 이 위치한 layer 와 연관된다. 좋은 대안은 없다

Iteration 2 — Step 4: Design Concept 선택 (2)

- Framework 선택은 Externally Developed Component 라는 Design Concept 에 해당한다 (CRN-2 활용)

설계 결정	근거(Rationale)
Spring framework 와 Hibernate 사용	Spring 은 엔터프라이즈 애플리케이션 개발을 지원하는 널리 쓰이는 framework 이고, Hibernate 는 Spring 과 잘 통합되는 ORM(object-relational mapping) framework 이다
버린 대안: JEE	Spring 이 더 "경량" 이고 개발 팀이 이미 익숙해서 더 빠르고 높은 생산성을 기대할 수 있어 Spring 을 선택
버린 대안: 다른 ORM framework	개발 팀이 Hibernate 에 이미 익숙하고 그 성능에 만족하고 있음

Iteration 2 — Step 5: 요소 인스턴스화와 책임 할당

■ 인스턴스화 설계 결정

- 초기 domain model 만 만든다
- 시스템 use case 를 domain object 에 대응시킨다
- Domain object 를 layer 별로 분해해, 명시적 인터페이스를 가진 layer 별 module 을 식별한다
- Module 에 대응하는 component 들을 Spring 으로 연결한다
- Framework 를 data layer 의 module 과 연관시킨다

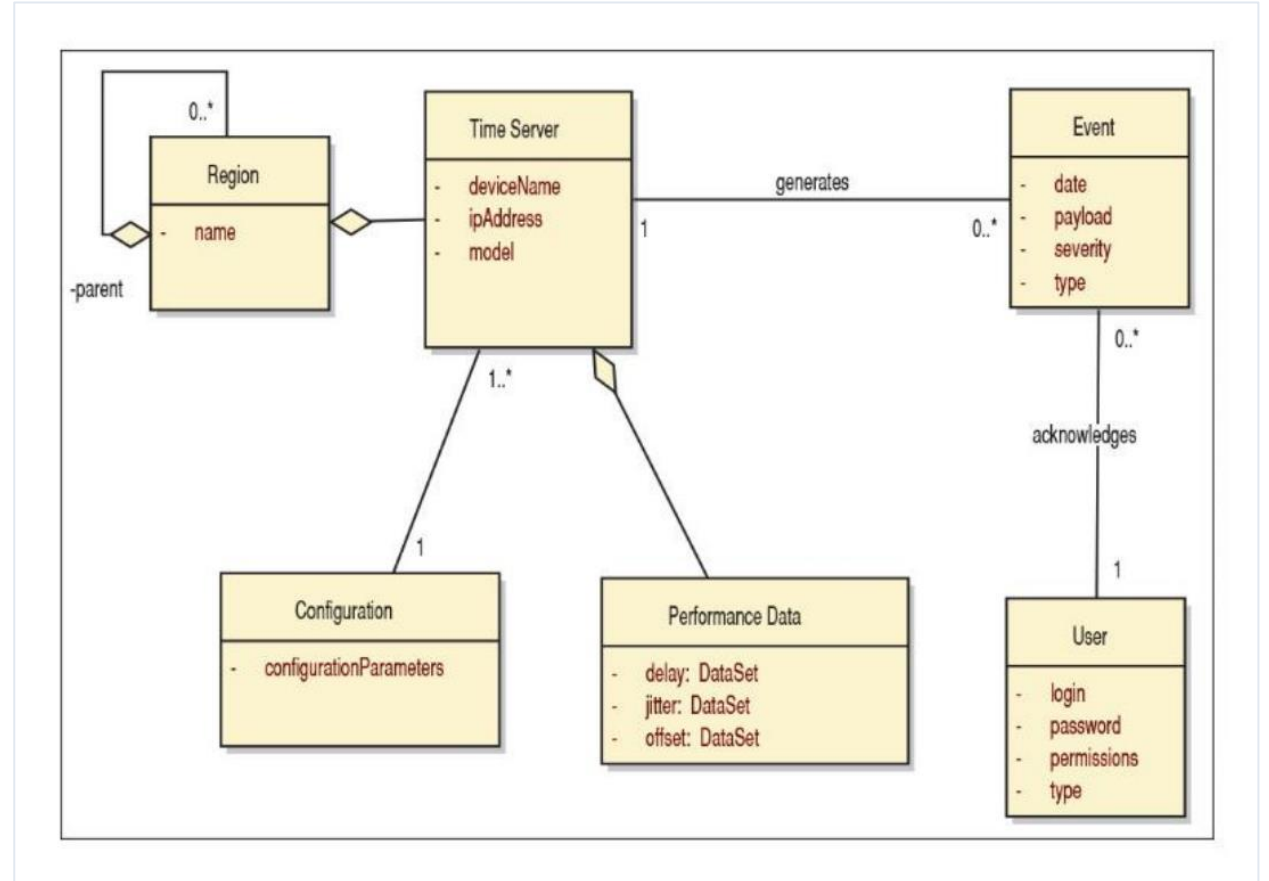
■ 참고

- 구조와 인터페이스는 이 단계에서 식별되지만, 실제로 표현(기록)하는 것은 다음 단계(Step 6)에서 한다

Iteration 2 — Step 6: 초기 Domain Model

■ 주요 entity

- Time Server(deviceName, ipAddress, model)가 Region 에 속하고 Event 를 생성한다
- Time Server 는 Configuration 과 Performance Data(delay, jitter, offset)를 가진다
- User(login, password, permissions, type)가 Event 를 확인(acknowledge)한다

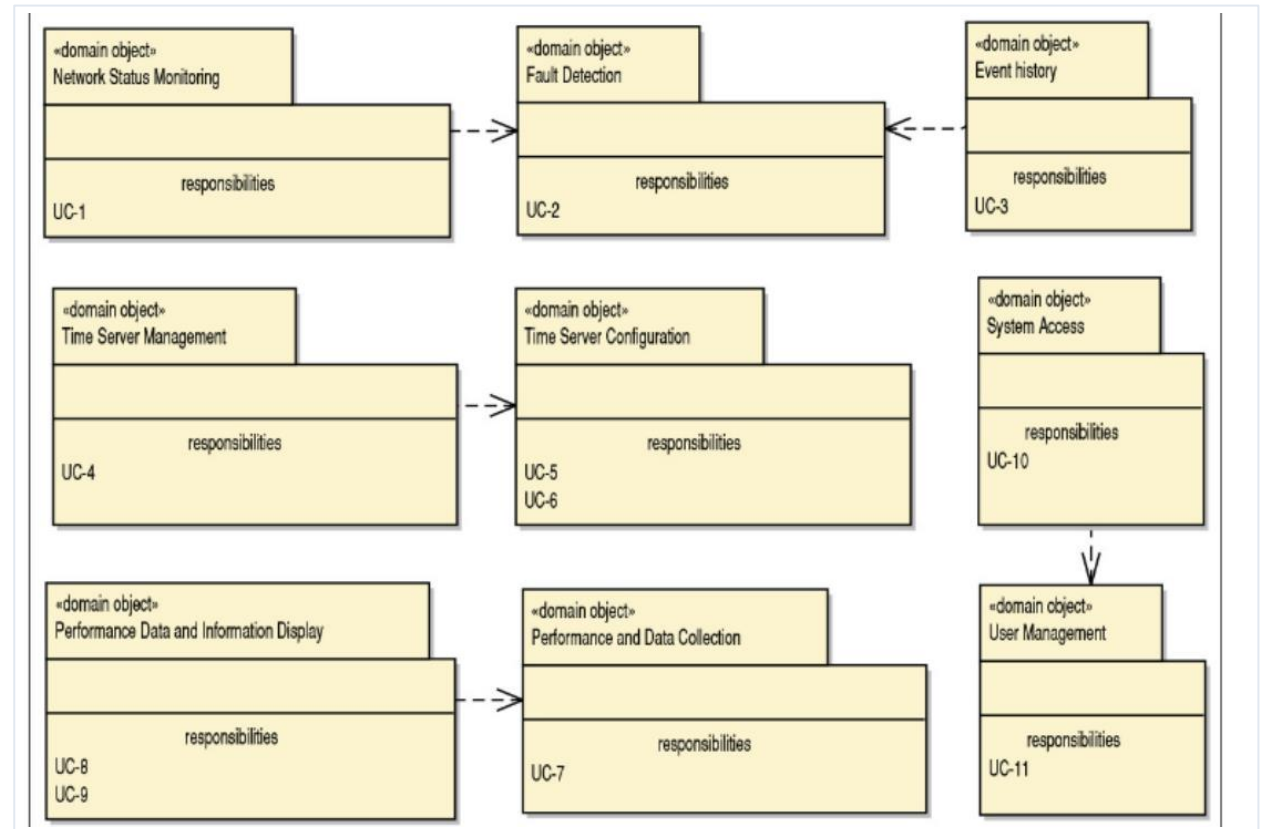


초기 Domain Model

Iteration 2 — Step 6: Use Case 별 Domain Object

■ Use case model 에 대응해 인스턴스화한 domain object

- Network Status Monitoring(UC-1), Fault Detection(UC-2), Event History(UC-3)
- Time Server Management(UC-4), Time Server Configuration(UC-5, 6), System Access(UC-10)
- Performance Data and Information Display(UC-8, 9), Performance and Data Collection(UC-7), User Management(UC-11)

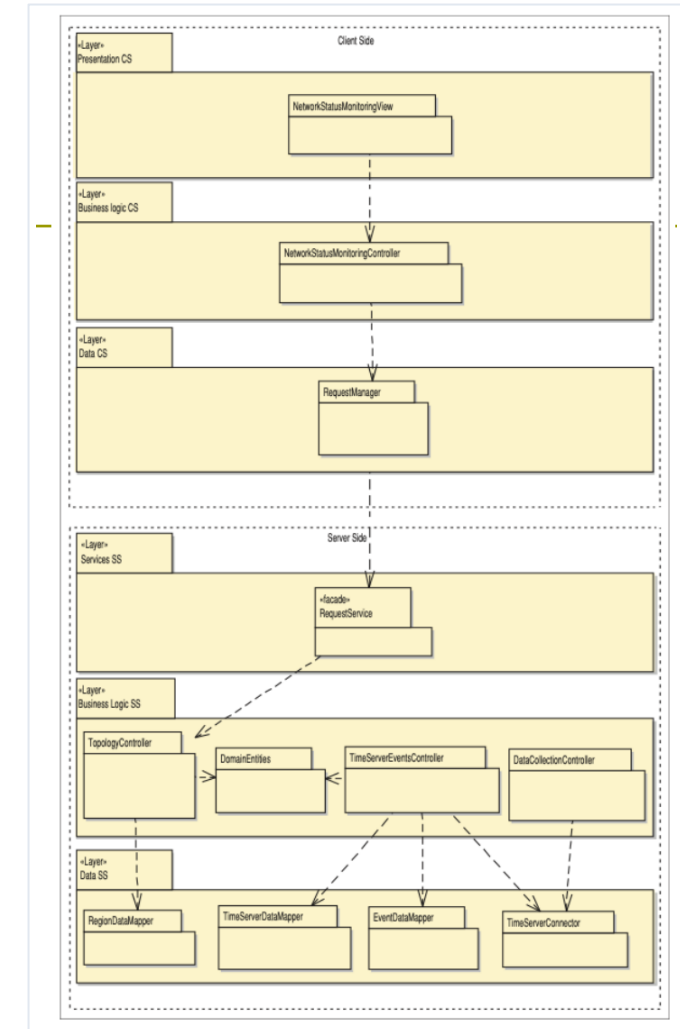


Use Case Model 에 대응하는 Domain Object 들

Iteration 2 — Step 6: Module View 스케치

■ Domain object 에서 도출한 module

- Domain object 에서 도출되어 primary use case 와 연관된 module 들로 구성된 module view
- Client Side: NetworkStatusMonitoringView → NetworkStatusMonitoringController → RequestManager
- Server Side: RequestService → TopologyController, TimeServerEventsController, DataCollectionController → DataMapper 와 TimeServerConnector
- 명시적 인터페이스는 그림에 나타내지 않았지만 존재한다고 가정한다



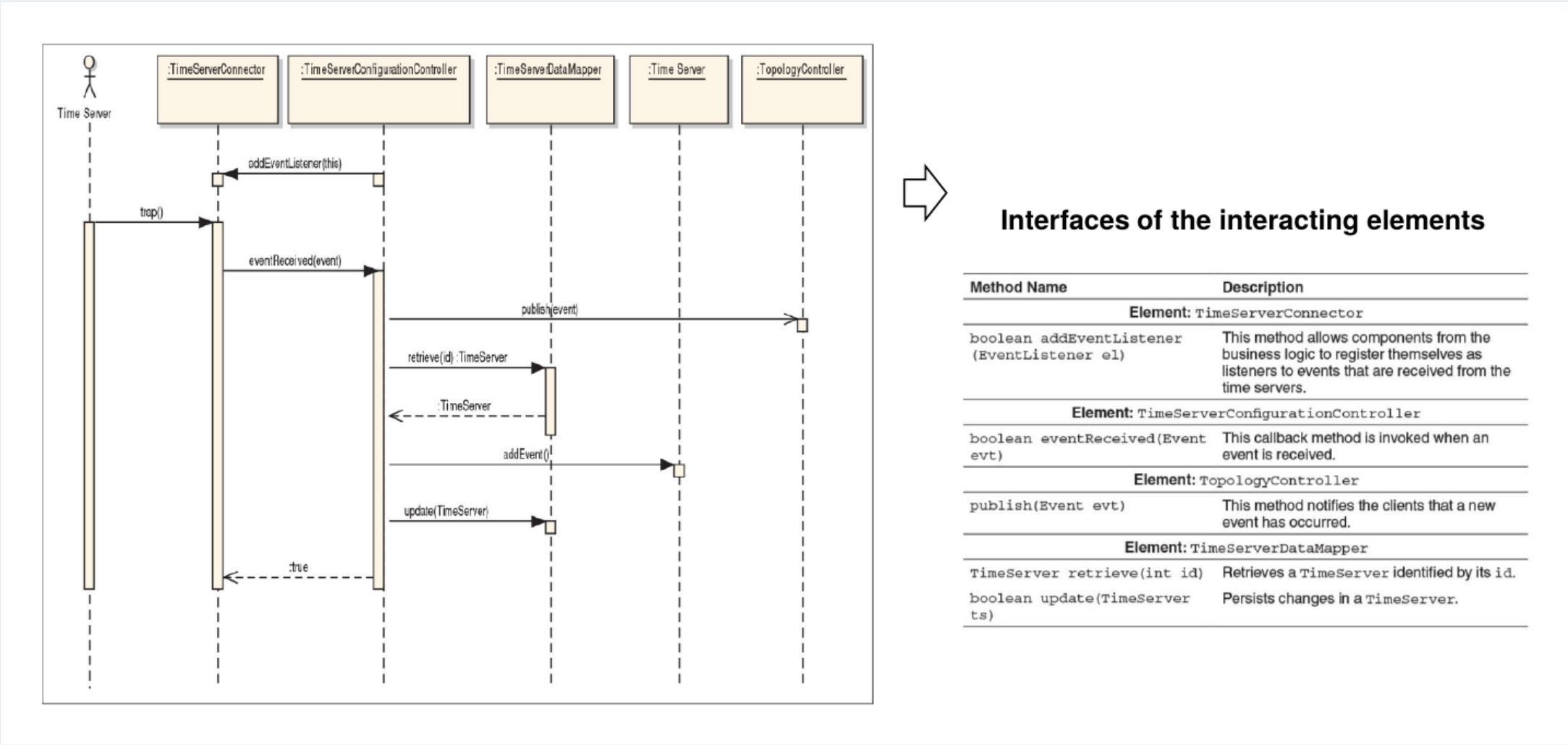
Iteration 2 의 Module View 스케치

Iteration 2 — Step 6: UC-2 Sequence Diagram 과 인터페이스

- 상호작용하는 요소들의 인터페이스를 정의하기 위해 UC-2(장애 탐지)의 sequence diagram 을 작성했다

Element	Method	설명
TimeServerConnector	boolean addEventListener(EventListener el)	business logic 의 component 가 time server 에서 오는 event 의 listener 로 자신을 등록한다
TimeServerConfigurationController	boolean eventReceived(Event evt)	event 가 수신되면 호출되는 callback method
TopologyController	publish(Event evt)	새 event 가 발생했음을 client 에게 알린다
TimeServerDataMapper	TimeServer retrieve(int id) / boolean update(TimeServer ts)	id 로 TimeServer 를 조회한다 / TimeServer 의 변경을 저장한다

Iteration 2 — Step 6: UC-2 Sequence Diagram



UC-2 의 Sequence diagram 과 상호작용 요소의 인터페이스

Iteration 2 — Step 7: 분석과 검토

Driver	상태	이번 iteration 의 설계 결정
UC-1	완전 해결	이 use case 를 지원하는 layer 별 module 과 예비 인터페이스를 식별
UC-2	완전 해결	이 use case 를 지원하는 layer 별 module 과 예비 인터페이스를 식별
UC-7	완전 해결	이 use case 를 지원하는 layer 별 module 과 예비 인터페이스를 식별
QA-1	부분 해결	연관된 use case(UC-2)를 지원하는 요소를 식별
QA-2	부분 해결	연관된 use case(UC-5)를 지원하는 요소를 식별
QA-3	부분 해결	관련 결정 없음
QA-4	부분 해결	연관된 use case(UC-7)를 지원하는 요소를 식별
CON-1	부분 해결	관련 결정 없음
CON-4	부분 해결	관련 결정 없음

Iteration 3 — Step 2·3: 목표 수립과 정제 요소 선택

■ Iteration 3 의 방향

- Iteration 1, 2 에서 내린 기본 구조 결정을 바탕으로, 이제 중요한 Quality Attribute 의 충족 여부를 따져 볼 수 있다
- 이 iteration 은 QA Scenario 하나에만 집중한다

■ Step 2: Iteration 목표 — QA-3 (Availability)

- "운영 중 관리 시스템에 장애가 발생하면, 관리 시스템은 30초 이내에 운영을 재개한다"

■ Step 3: 정제할 요소

- 이 Availability scenario 를 위해, 첫 iteration 에서 식별한 물리 node 를 정제한다
 - Application server
 - Database server

Iteration 3 — Step 4: Design Concept 선택

- Tactic(Active Redundancy)과 Externally Developed Component(Message Queue) 두 종류의 Design Concept 을 사용한다

설계 결정과 위치	근거와 가정
Application server 와 database 같은 핵심 component 를 복제하는 Active Redundancy tactic 도입	핵심 요소를 복제하면, 복제된 요소 중 하나가 고장 나도 기능에 영향 없이 시스템이 버틸 수 있다
Message Queue 기술 계열의 요소 도입	Time server 에서 받은 trap 을 message queue 에 넣고 애플리케이션이 꺼내 가도록 한다. Queue 를 쓰면 trap 이 순서대로 처리·전달되는 것을 보장할 수 있다(QA-1)

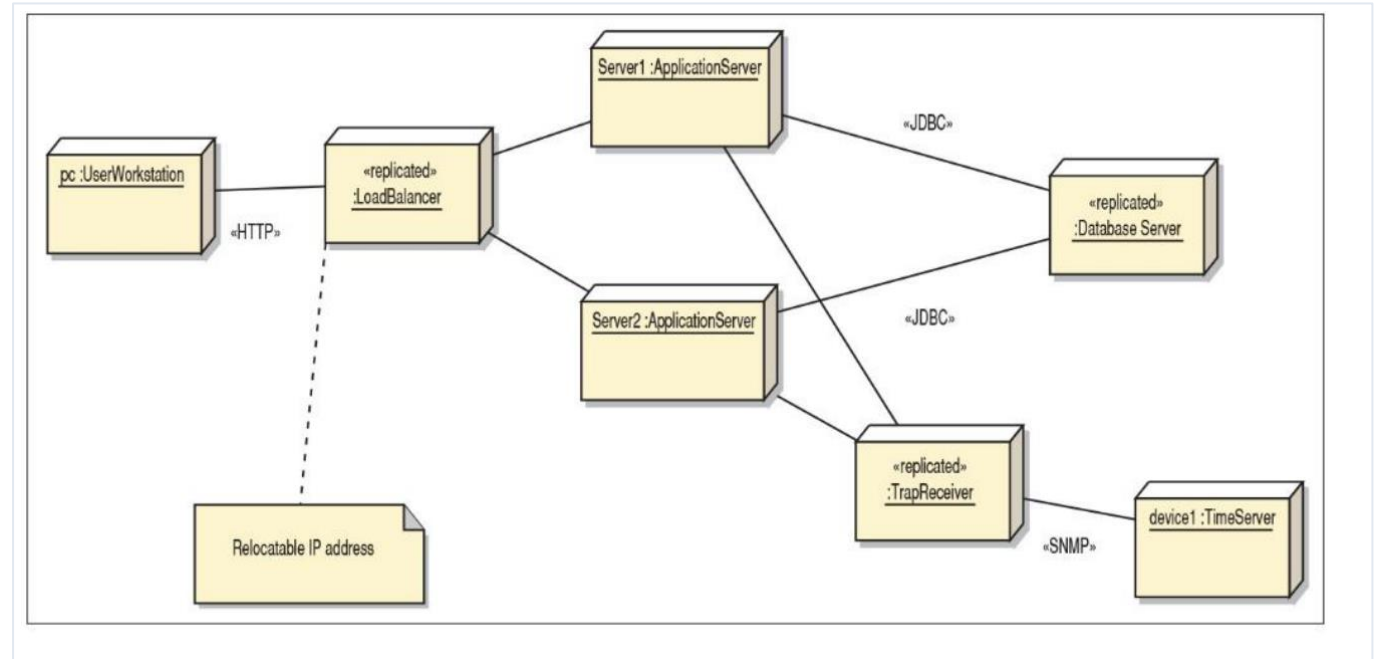
Iteration 3 — Step 5: 요소 인스턴스화와 책임 할당

설계 결정과 위치	근거(Rationale)
Message queue 를 별도 node 에 배포	별도 node 에 message queue 를 두면 애플리케이션 장애 시에도 trap 을 잃지 않는다. 이 node 는 active redundancy tactic 으로 복제하되, 한 복제본만 네트워크 장비에서 오는 event 를 받아 처리한다
Application server 에 active redundancy 와 load balancing 사용	Application server 복제본 두 개가 동시에 활성 상태이므로, 그 사이에 부하를 분산하는 것이 합리적이다. Load-Balanced Cluster pattern 으로 달성한다. 새 Architectural Concern CRN-5(복제본의 상태 관리)가 생긴다
기술 지원을 이용해 load balancing 과 redundancy 구현	Load balancing 과 redundancy 를 위한 기술 대안이 많으므로, 덜 성숙하고 지원하기 어려운 ad hoc 해법을 직접 만들 필요가 없다

Iteration 3 — Step 6: 정제된 Deployment Diagram

■ Redundancy 가 도입된 배포 구조

- Workstation → LoadBalancer(replicated)
→ Server1 / Server2
- Application server → Database Server(replicated)
- TrapReceiver(replicated) ← Time Server(SNMP)
- Load balancer 는 Relocatable IP address 를 사용한다



Iteration 3 의 정제된 Deployment diagram

Iteration 3 — Step 7: 분석과 검토 (1) QA

Driver	상태	이번 iteration 의 설계 결정
QA-1	부분 해결	별도의 복제된 trap receiver node 를 도입해, application server 에 장애가 나도 trap 의 100% 를 처리할 수 있다. 또한 trap 수신을 별도 node 가 맡으므로 application server 의 처리 부하가 줄어 성능에도 도움이 된다. 구체적 기술은 아직 정하지 않아 "부분 해결"
QA-2	부분 해결	관련 결정 없음
QA-3	부분 해결	Application server 를 이중화해 시스템 장애 확률을 낮춘다. Load balancer 가 고장 나면 passive replica 가 요구 시간 안에 활성화된다. 구체적 기술(message queue)은 아직 정하지 않아 "부분 해결"
QA-4	부분 해결	관련 결정 없음

Iteration 3 — Step 7: 분석과 검토 (2) CON·CRN

- 설계를 진행하다 보면 새로운 Concern(CRN-5)이 생기기도 하며, 이는 다음 iteration 의 Driver 가 된다

Driver	상태	이번 iteration 의 설계 결정
CON-1	부분 해결	Application server 복제와 load balancer 사용이 다수 사용자 요청 지원에 도움이 된다
CON-4	부분 해결	관련 결정 없음
CON-5	부분 해결	관련 결정 없음
CON-6	부분 해결	관련 결정 없음
CRN-2	부분 해결	관련 결정 없음
CRN-4	부분 해결	관련 결정 없음
CRN-5	미해결	이번 iteration 에서 새로 도입된 Architectural Concern: 복제본의 상태 관리. 아직 관련 결정 없음

요약 (Summary)

■ Architecture 설계

- Architectural Design 은 주어진 요구사항을 만족하는 일련의 설계 결정을 내려 Architecture 를 만드는 활동이다
- 설계 결정을 내리기 전에 가능한 설계 대안을 탐색하고 평가해야 한다

■ Architectural Drivers

- Architecture 의 모양을 결정하는 요구사항
 - Primary Functionality, Quality Attributes, Constraints, Design Purpose, Architectural Concerns

■ Design Concepts

- 구조를 만드는 데 사용하는 재사용 가능한 building block
 - Reference Architecture, Architectural/Design Pattern, Deployment Pattern, Tactic, Externally Developed Component

■ Attribute-Driven Design (ADD)

- Quality 요구사항, 기능 요구사항, Constraint 를 모두 만족하는 Architecture 를 설계하는 반복적(iterative) 접근법

핵심 Driver 를 정하고 → Design Concept 을 골라 → 결정과 근거를 기록하며 → 반복한다